STAT GR5293 Sec 002
Spring 2026

Generative AI using Large Language Models (LLMs)

Lecture 1 01/23/26
Fr 4:10pm - 6:40pm
517 HAMILTON HALL

Instructor

Chen Wang

✉ <cw3687@columbia.edu>

🏛 **Adjunct Associate Professor, Computer Science Department**

- Machine learning, deep learning, GenAI Inference & Serving
- LLM Model Finetuning

Senior Research Scientist at IBM Research, NY

- Kubernetes, Container Cloud Platform,
- Data-driven/QoE based resource management,
- AI4Sys & Sys4AI, LLM Serving & Finetuning
- Sustainable Cloud & AI systems

Class Introduction

Prerequisites

Knowledge of Machine Learning and Deep Learning

Programming in Python

Course Resources & Information

All information about the class will be available here including syllabus, announcements and assignments.

Access Class on CourseWork

https://courseworks2.columbia.edu/courses/239608

Today's Agenda

Course Overview

Syllabus

Assignments and Grading

Logistics

Deep Learning fundamentals

ML & DL Fundamentals

- Basic Concepts
- Training Dynamics
- Hyperparameters

Hardware & Distributed Training

- Hardware Connection
- Parallelism
- Synchronization

Course Information

What this course will cover ?

- ML concepts, DL overview, Transformers, LLMs
- LLM pre-training and fine-tuning
- Prompt engineering, fine-tuning, and LLM application development
- Retrieval Augmented Generation (RAG)
- LLM benchmarking and performance evaluation
- Tool-assisted LLMs, LLM based agents
- Generative AI on cloud platforms
- RLHF
- LLM Serving
- Multi-modality
- Research paper readings

What this course will not cover ?

- Details about specific ML and DL algorithms

Class 1: Deep Learning (DL) Fundamentals

ML Performance Concepts/Techniques

- Overfitting, generalization, bias, variance tradeoff, regularization
- Performance metrics: algorithmic and system Level

DL Training Fundamentals

- Backpropagation, activation functions
- Data preprocessing, batch normalization
- SGD and its variants
- Exploding and vanishing gradients
- Weight initialization

DL Training Hyperparameters & Regularization

- Batch size, learning rate, momentum, weight decay
- Regularization techniques in Deep Learning

System & Hardware

- Single Node vs Distributed Training
- Model, Data, and Hybrid Parallelism
- Convergence & Runtime
- Hardware Accelerators

Class 2: Attention, Transformer, and Popular Large Language Models (LLMs)

Core Architectures & Mechanisms

- Seq2Seq models
- Encoder and decoder
- Attention mechanism
- Transformer architecture
  - self-attention
  - multi-head attention
  - encoder-decoder attention

Popular LLMs

- BERT
- OpenAI GPT
- LLAMA
- Gemini
- Claude

Class 3: Prompt Engineering and LLM Apps

Exploring Techniques, Tools, and Applications

Prompting Techniques & Tools

- Zero-shot Prompting
- Few-shot Prompting
- Chain-of-Thought, Automatic CoT
- LangChain
- LlamaIndex

Basics, Use Cases & Development

Basics of Prompting

- Prompt Elements
- General Tips
- Examples

LLM Use Cases

- Translation
- Code Generation
- Summarization
- Proofread and Correct
- Math Calculation
- Entity Extraction
- Call Functions

Prompt Engineering & App Development

- Prompt Engineering Techniques
- LLM App Development

Class 4: RAG, Functions, and Tools

Capabilities & Limitations of LLM

- Knowledge Cutoffs
- Hallucinations
- Structured Data Challenge
- Biases

Retrieval Augmented Generation

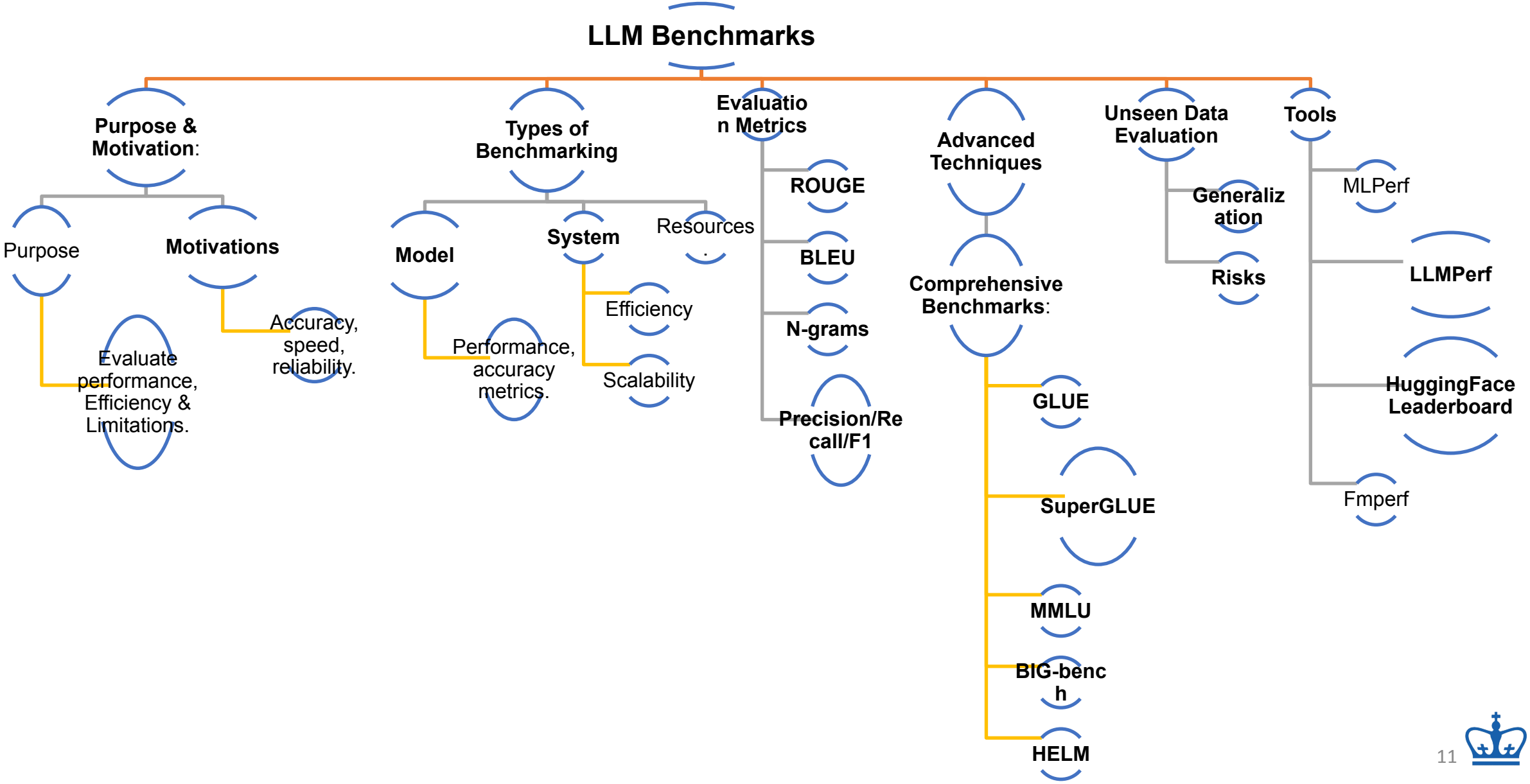
- Use Cases
  - Semantic Search
  - Summarization
- Keyword Search and Embeddings
- Retrieval and Rerank
- Answer Generation

Vector Databases

Class 5: Generative AI using Cloud

Key Topics	Core Concepts	Practical Skills	Hands-on Experience	Learning Outcomes
• Google AI Studio • AWS Generative AI • LiteLLM for unified LLM access	• Generative AI application lifecycle Foundation model selection and customization Prompt design and model tuning Retrieval Augmented Generation (RAG)	• Prototyping with Vertex AI Studio Implementing multimodal applications Evaluating model performance Deploying AI solutions	• Local installation of LiteLLM Accessing various LLM APIs (e.g., AWS Bedrock, Anthropic, Cohere, Hugging Face) Writing code for model interaction	• Understand cloud-based AI tools and their applications in statistics Gain practical experience with industry-standard GenAI platforms Develop skills to leverage diverse LLMs for statistical problems

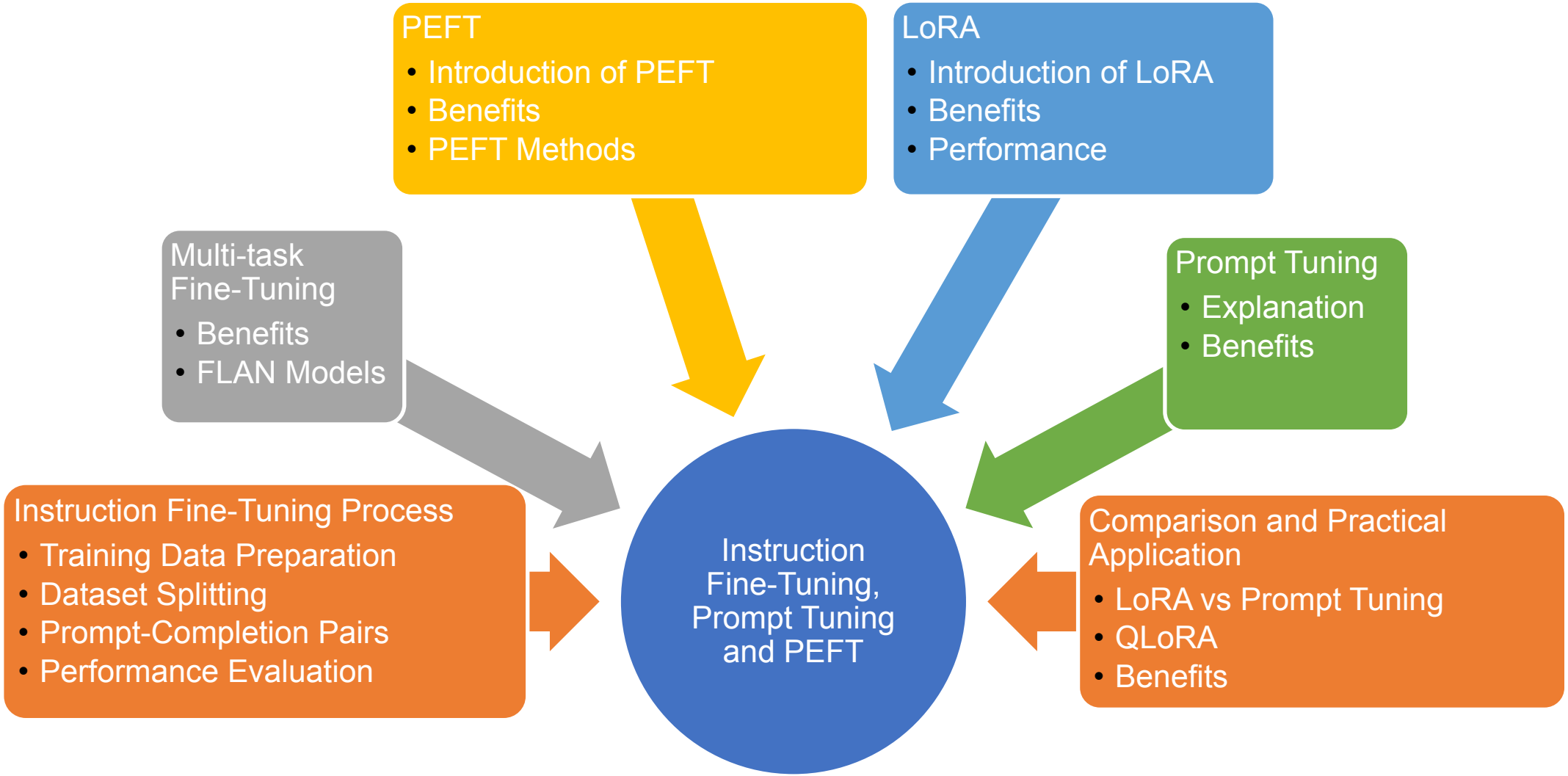
Class 6: LLM Benchmarking



Class 7: Pre-Training for LLM

Pre-training Concepts	Training Process for Different Architectures	Managing High Memory Requirements	Scaling Model Training	Optimizing Training Resources	Use Cases for Custom LLM Pre-training
1. Training from existing foundation models 2. Training from scratch 3. Model selection from HuggingFace and PyTorch hubs	1. Encoder-only models 2. Decoder-only models 3. Sequence-to-sequence (seq-to-seq) models	1. Quantization techniques 2. Challenges with consumer-grade hardware	1. Distributed Data Parallel (DDP) 2. Fully Sharded Data Parallel (FSDP) 3. Zero Redundancy Optimizer (ZeRO)	1. Balancing model size, training data volume, and compute budget 2. Insights from the Chinchilla study	1. Domain adaptation (e.g., law, medicine) 2. Introduction to BloombergGPT as an example

Class 8: Fine Tuning Techniques

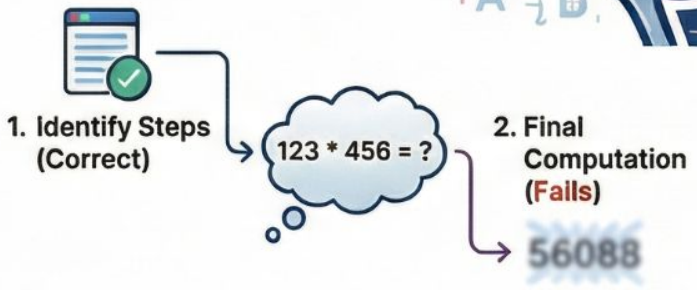


Class 9: Tool-assisted LLMs and Agentic AI

The Problem: Linguistic Logic vs. Mathematical Precision

LLMs Struggle with Precise Calculations

Models like Llama-2 often hallucinate incorrect results for large-number multiplication tasks.

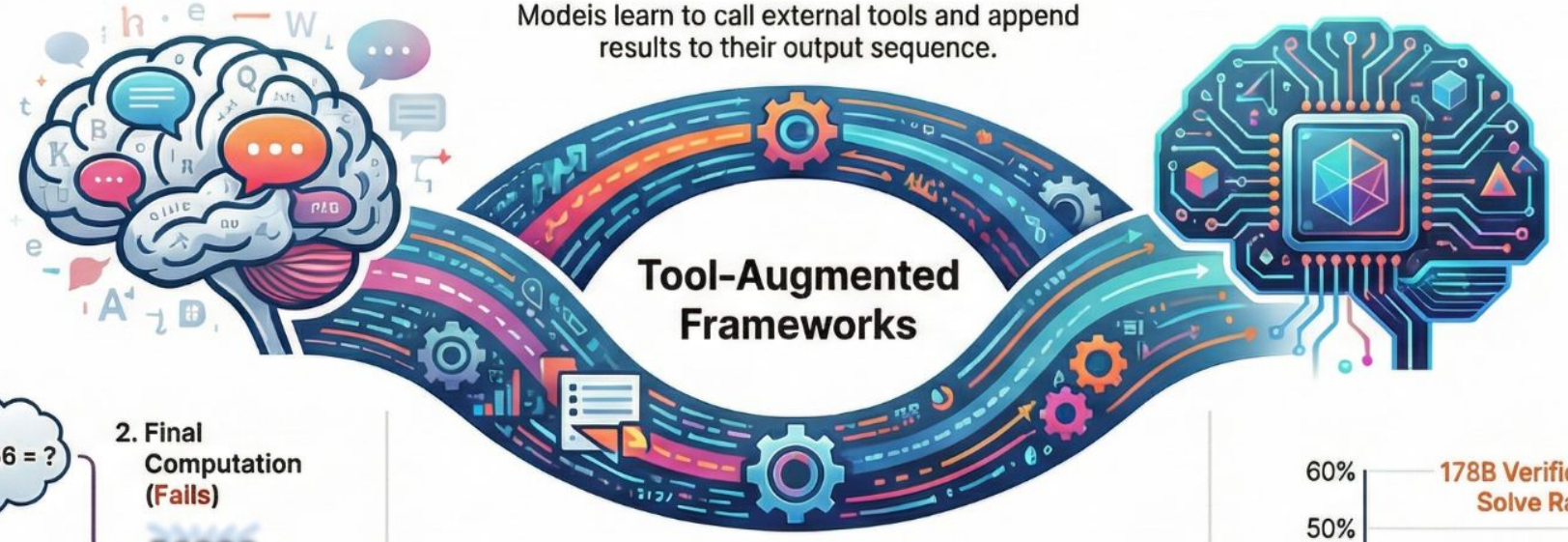


The Logic-Calculation Disconnect

LLMs may correctly identify a math problem's steps but fail the final computation.

TALM: Tool Augmented Language Models

Modeis learn to call external tools and append results to their output sequence.



PAL: Program-aided Language Models

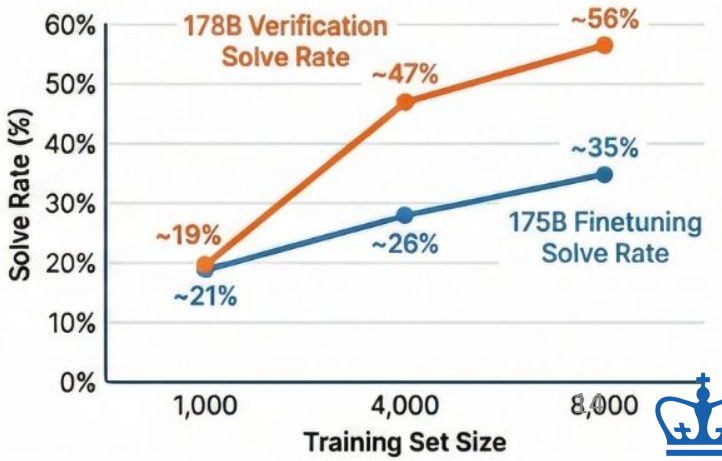
LLMs offload reasoning to a programmatic runtime (like Python) to ensure logical accuracy.

The Solution: Tool-Augmented Frameworks

Verification Boosts Performance

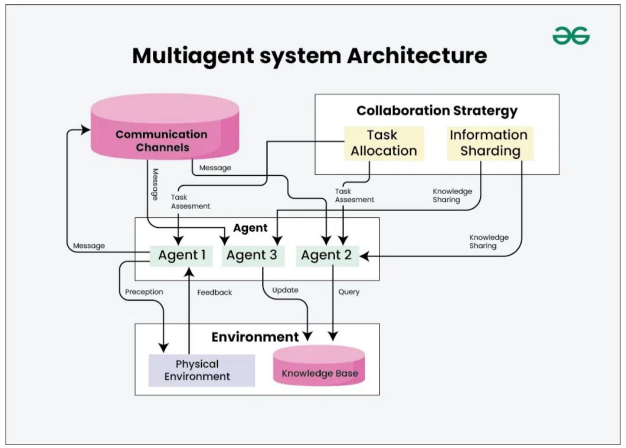
Comparison of Performance on GSM8K Dataset

Significantly increases solve rates compared to standard finetuning as data scales.



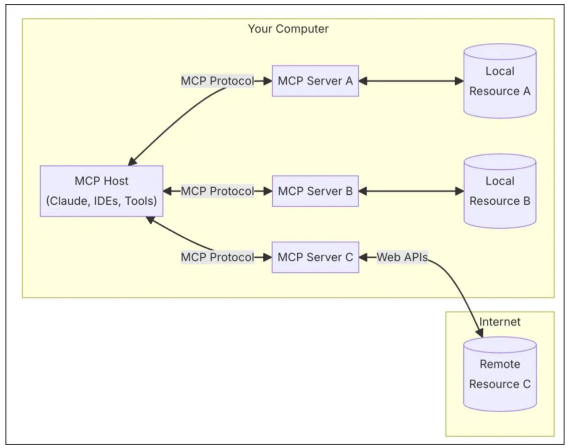
Class 10: Multi-agent systems and MCP

Multi-agent Systems



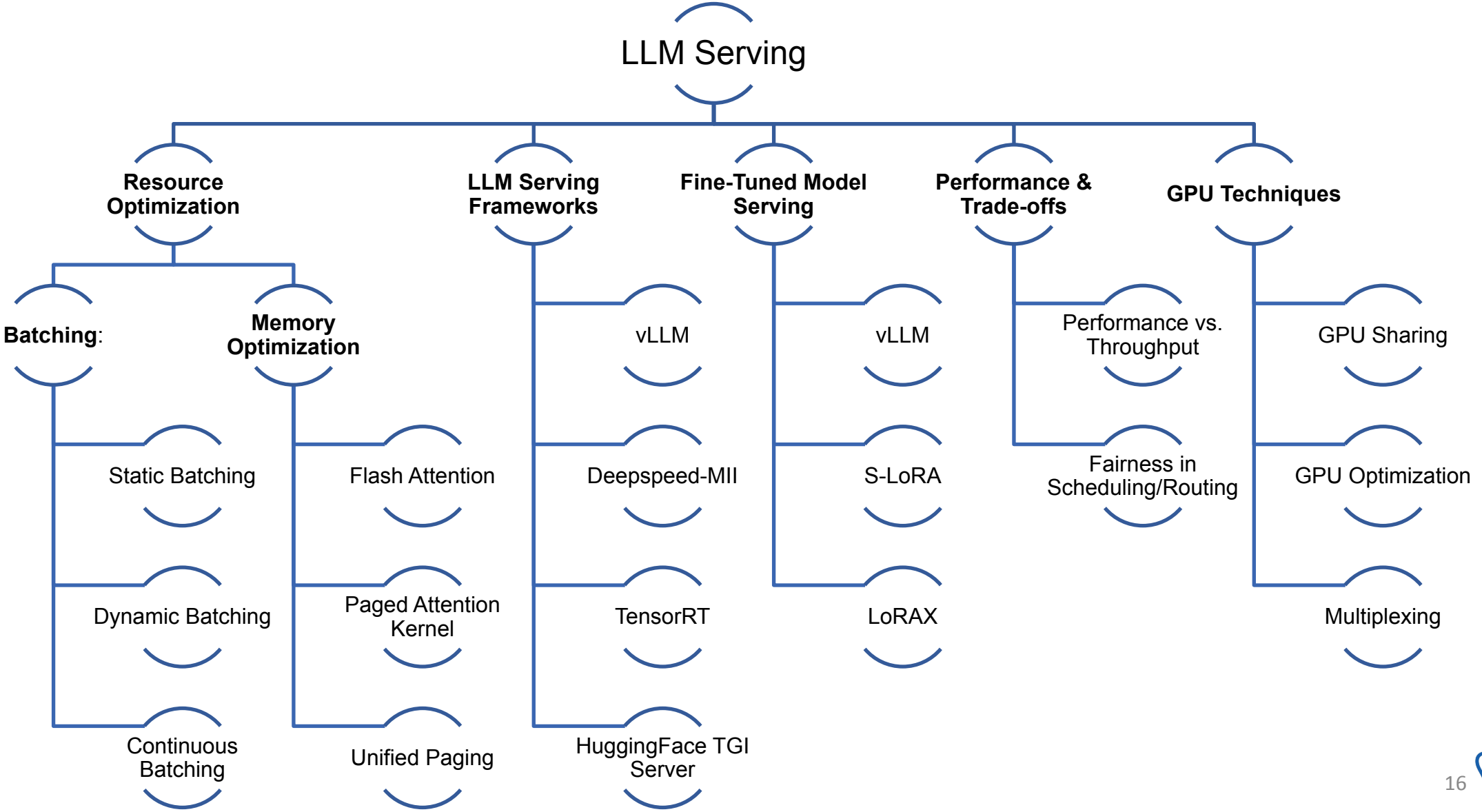
- A pool of specialized agents collaborating to solve complex tasks
- Benefits: modularity, specialization, and control in agentic AI systems
- We'll study connection patterns between agents
- Frameworks: LangGraph and CrewAI

Multi-Context Protocol (MCP)



- Framework enabling LLMs to efficiently manage multiple conversations
- Improves performance and reduces computational costs
- We'll explore MCP clients and servers development
- Practical implementations for optimizing LLM interactions

Class 11: Efficient Serving of LLMs



Class 12: RLHF

RLHF

Introduction	**Concept**: Human feedback guides reinforcement learning.
	Purpose: Align LLMs with human values/preferences.
Fine-Tuning Methods	**Instruction Fine-Tuning**: Tailor LLMs to follow specific instructions.
	Path Methods: Guide learning based on human feedback.
Challenges in RLHF	**Toxicity/Misinformation**: Mitigate harmful content.
	Human Alignment: Ensure helpful, honest, harmless models.
RLHF Process	**Reward Model Training**: Evaluate outputs, assign rewards.
	Automated Labelers: Replace human labelers, select completions.
	LLM Updating: Align models with human feedback.
Proximal Policy Optimization (PPO)	**PPO Overview**: Fine-tuning via reinforcement learning.
	RLHF Application: Iterative updates, maximize rewards.
Constitutional AI	**Concept**: Ethical AI alignment with human values.
	Role in RLHF: Ensure adherence to societal norms, trust, safety.

Class 13 Voice AI Agent

1. Core Concepts & Architectures

- **Define AI Voice Agents:** Systems combining foundation model reasoning with speech abilities for human-like, real-time interactions.
- **Compare Architectural Approaches:**
  - **Pipeline Approach:** Multi-step process (VAD → STT → LLM → TTS).
  - **S2S (Speech-to-Speech) Approach:** "Real-Time" model approach (e.g., Gemini Multimodal Live), simpler implementation but less granular control.

2. The Voice Agent Stack

- **Breakdown of essential components:**
  - **VAD & EOU:** Detecting speech presence and determining speaker turn completion.
  - **STT (ASR):** Transcribing audio signals into text for the LLM.
  - **LLM Reasoning:** Generating responses and managing conversation context.
  - **TTS:** Synthesizing natural-sounding speech from text.

3. Solving the Latency Challenge

- Focus on "human-like" response times (~236 ms), while current AI stacks range from 540 ms to 1.6 s.
- **Optimization Strategies**:
  - **Streaming APIs**: Processing audio and text "token-by-token".
  - **Networking Protocols**: WebRTC (UDP-based) preferred for low-latency streaming, congestion control, and data compression.

4. Advanced Interaction Handling

- Moving beyond basic turn-taking for complex scenarios:
  - **Turn Detection**: Using transformer models and binary classifiers to manage speaker transitions.
  - **Interruption Handling**: Implementing logic to stop speaking immediately upon user interruption.

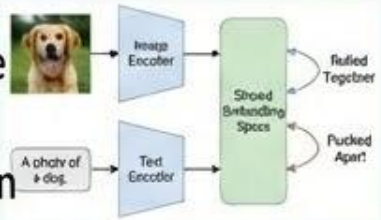
Class 14: Vision Language Models

Part 1: CLIP (Contrastive Language-Image Pre-training)

Revolutionized visual concept learning from natural language supervision.

Trained on 400 million image-text pairs using contrastive learning.

Achieves robust zero-shot transfer capabilities, often matching fully supervised baselines.

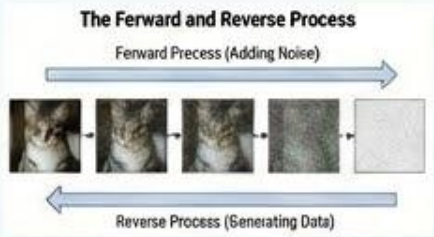


Zero-Shot Transfer: Can recognize unseen categories (e.g., "zebra") without explicit training data for that class.

Part 2: Diffusion Models

A new class of generative models that have recently outperformed GANs in creating high-fidelity images

They generate data by reversing process, moving from pure noise back to a structured image.

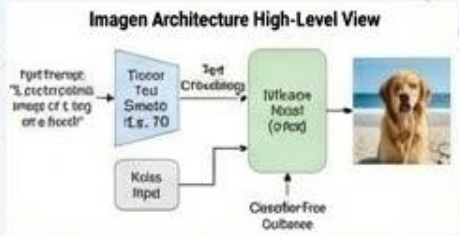


Reversing tha process invelves solving a reverse generative %lochasbe differentfel equation (SDE) or probability Row OOEs to reconstruct the image from noise.

Part 3: Imagen (Text-to-Image Architecture)

Combines concepts from CLIP and Diffusion Models. Leverages frozen text encoders (e.g., T5) to understand text prompts.

Uses classifier-free guidance to maximize photor-ealism and text alignment.



By using a powerful, pre-trained text encoder and a separate diffusion process wtth guidance, Imagen achieves state-of-the-art resolts.

Assignments and Grading

Distribution of Marks:

Assignments: 40% | Quizzes: 20% | Final Project: 40%

Assignments (40%)

- 5 assignments
- Posted at the end of lectures 2, 4, 6, 8, 10
- Due in 2 weeks
- All programming assignments as Jupyter notebooks (unless specified otherwise)

Quizzes (20%)

- 5 quizzes
- Conducted on Canvas

Final Project (40%)

- Team assignment
- Team of 2
- Any project involving performance of LLM systems

Class Logistics

Communication & Support

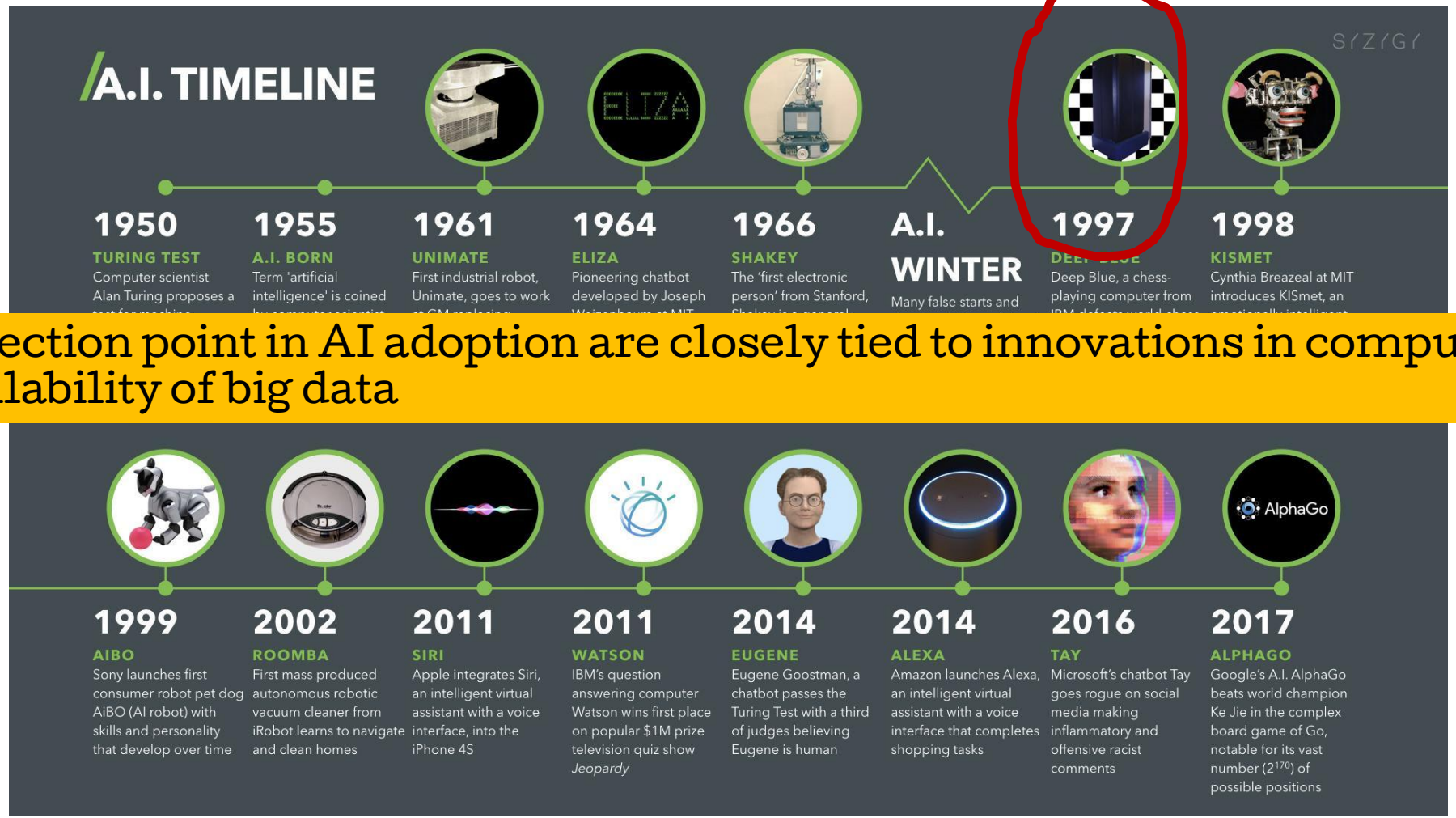
- Class communications: Ed
- TA Office hours: TBD

Access to Compute resources

- Google Colab (Free or Education Pro Subscription $9.9/month)
- Free GCP credits
- Free Gemini API access
- Free OpenAI API access
- Groq Free API key

The Genesis of the Data Age: 1997-2006

Cloud computing was coined



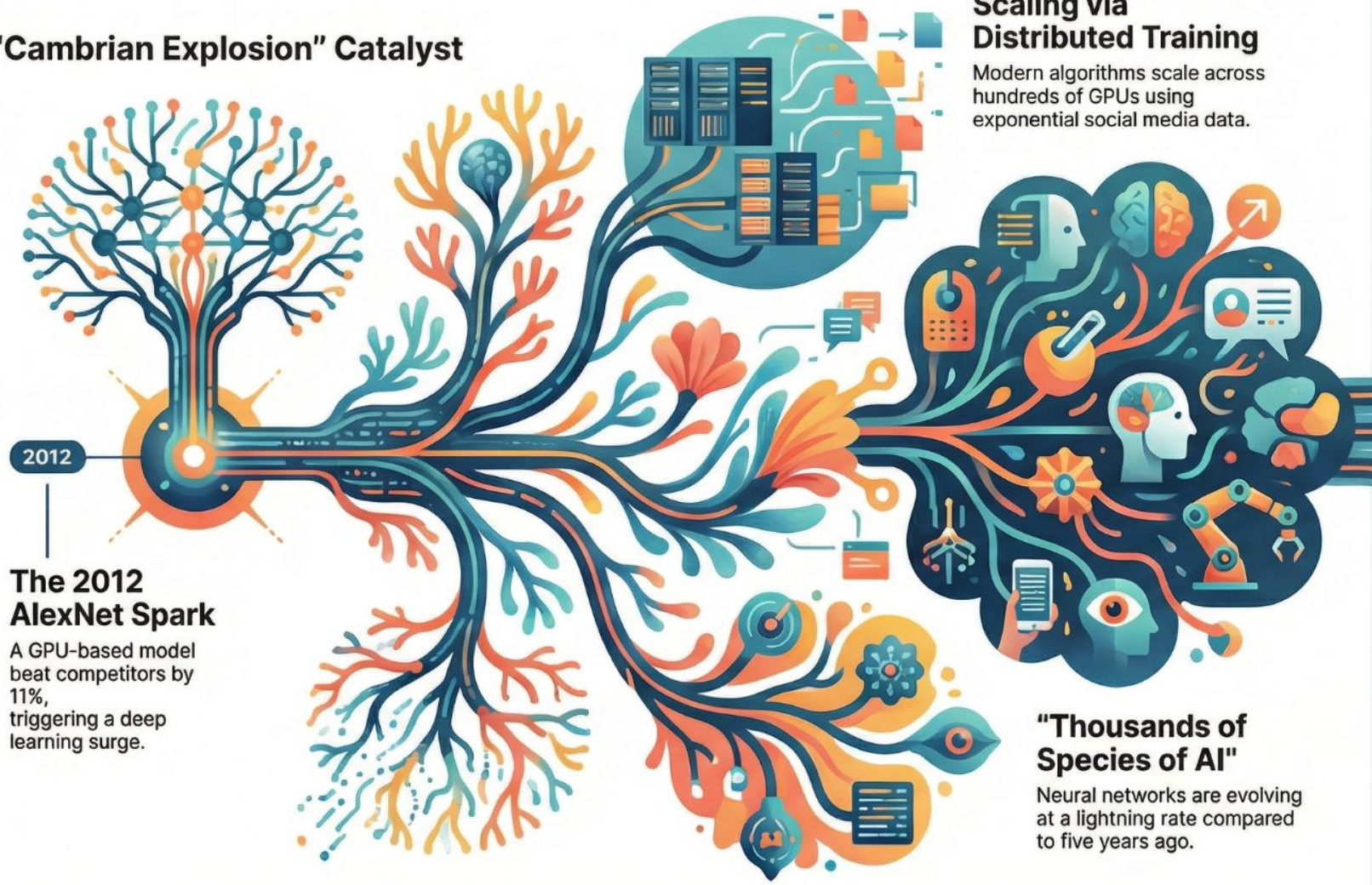
Inflection point in AI adoption are closely tied to innovations in computing and availability of big data

2006: Amazon elastic cloud and S3 was launched

The Artificial Intelligence Revolution: Drivers and Catalysts

The AI revolution is a convergence of critical drivers, catalyzed by a 2012 breakthrough leading to a "Cambrian Explosion" of diverse applications.

"Cambrian Explosion" Catalyst



The 2012 AlexNet Spark

A GPU-based model beat competitors by 11%, triggering a deep learning surge.

Scaling via Distributed Training

Modern algorithms scale across hundreds of GPUs using exponential social media data.

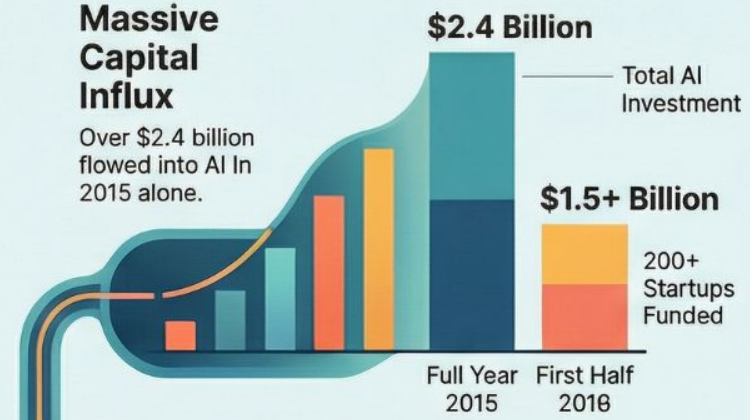
"Thousands of Species of AI"

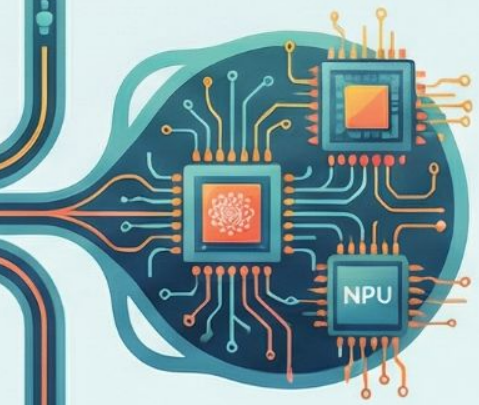
Neural networks are evolving at a lightning rate compared to five years ago.

The 7 Drivers of Adoption

Massive Capital Influx

Over $2.4 billion flowed into AI In 2015 alone.





Specialized Hardware Evolution

Custom chips like TPUs and NPUs enable faster, more powerful machine learning.

Human-Machine Symbiosis

Modern AI focuses on augmenting human skille through trust and responsibility.

Evolution of Large Language Models (LLMs)

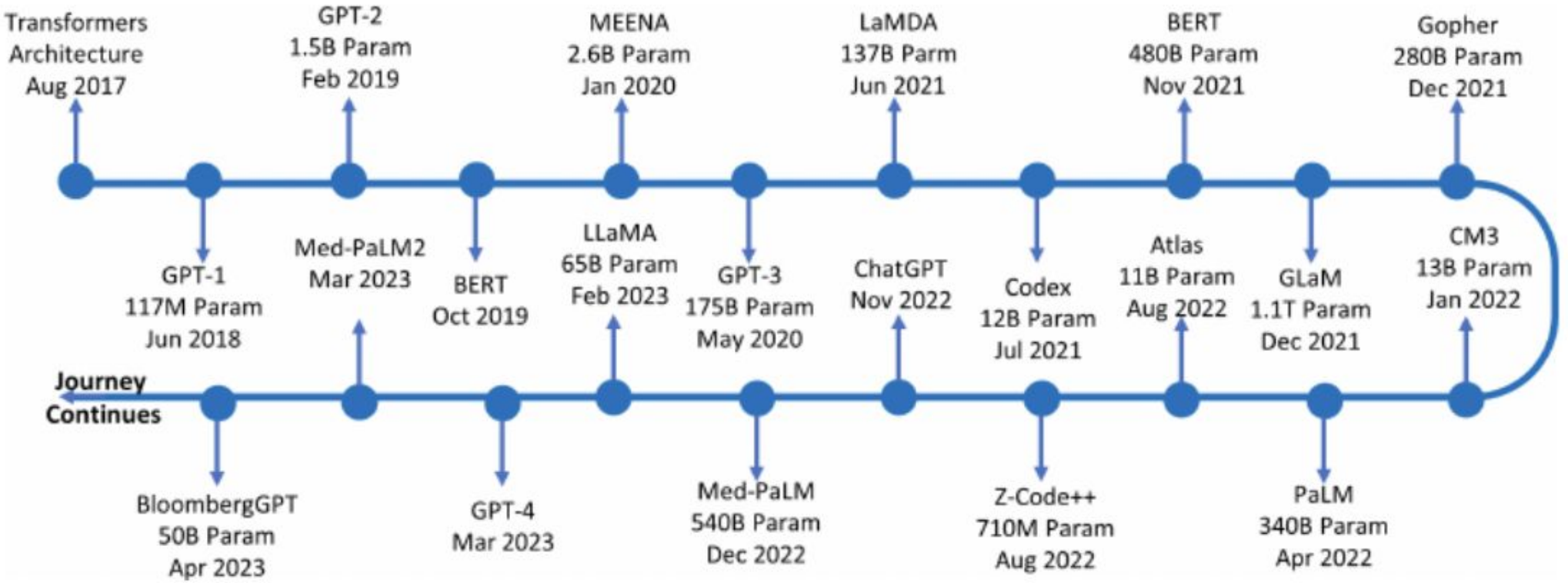


Figure from Mohamadi et al, ChatGPT in the Age of Generative AI and Large Language Models: A Concise Survey

AI Blogs of Major Companies

Meta AI

https://ai.meta.com/blog/

Google AI

https://ai.googleblog.com

IBM Research

https://www.ibm.com/blogs/research/category/ai/

Microsoft

https://news.microsoft.com/source/topics/ai/

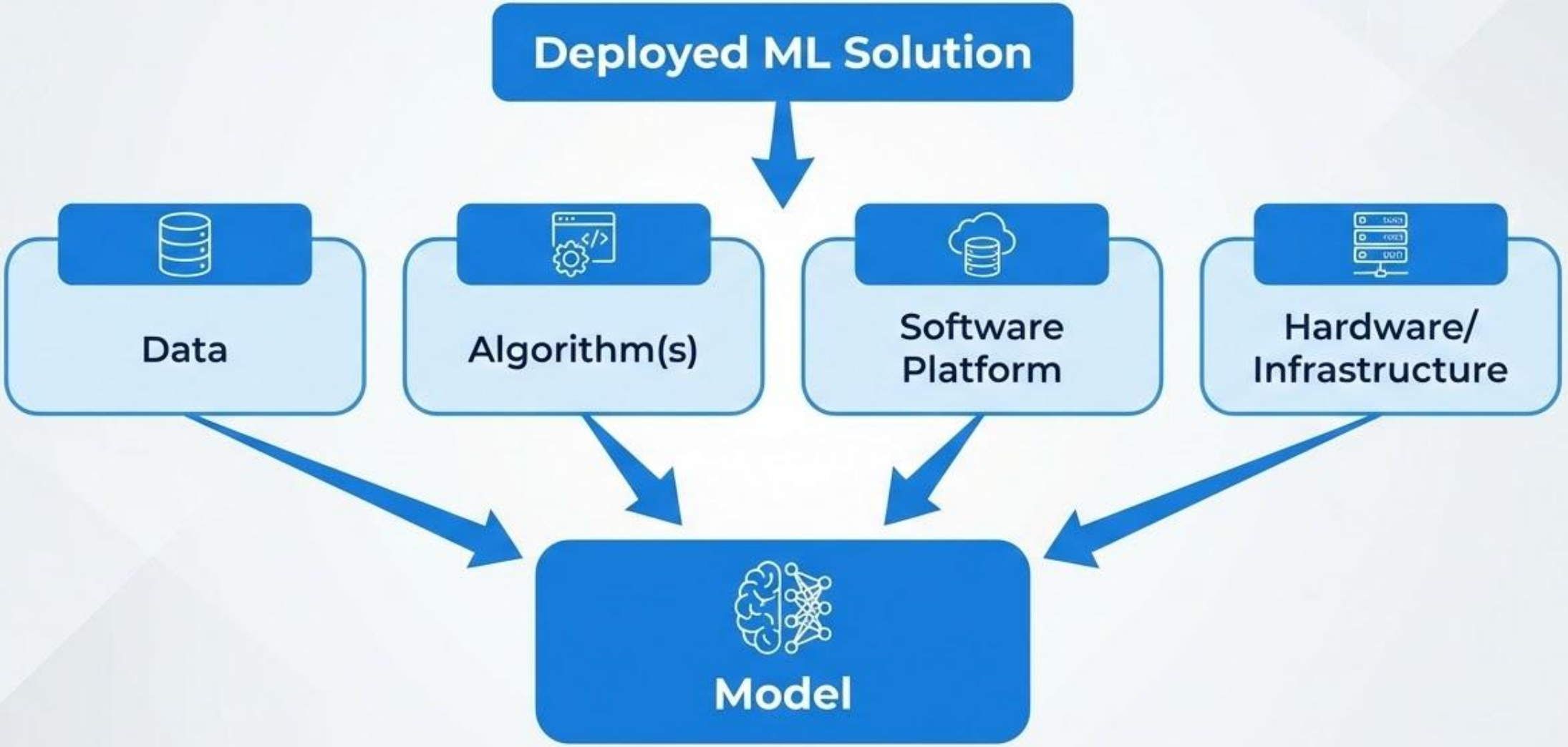
AWS Machine Learning Blog

https://aws.amazon.com/blogs/machine-learning/

xAI News

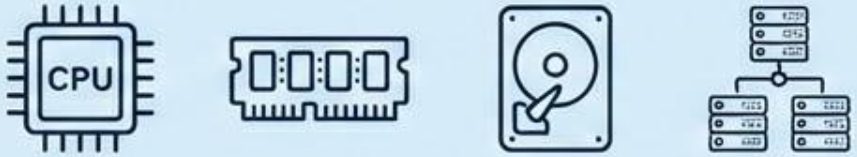
https://x.ai/news

Constituents of a Deployed ML Solution



Infrastructure

Core Components



Compute units & accelerators, memory, storage, network

Performance Impact

Hardware can help improve performance pretty much everywhere in the pipeline

Design Strategies

- Design better hardware
- Adapt existing architectures to ML tasks
- Develop brand-new architectures for ML

Precision Tradeoff

Hardware compute precision affects performance: tradeoff between accuracy and runtime

(Learning) Algorithm

Key Considerations

- General and domain specific architectures
- Hyperparameter tuning to extract the best performance performance
- Effects the resource requirements: compute throughput (FLOPS), memory

Performance & Scalability Dependencies

Performance (runtime) and scalability of an algorithm depends on:

Hardware/Infrastructure

Software platform (frameworks, libraries, drivers)

Data

The Critical Element

Data is the king in ML; determines choice of learning algorithm.

Modalities & Sources

Audio, video, images, text from diverse sources, collection, labeling, and quality.

Preparation Bottleneck

Making data business ready is challenging; 80% of time spent on preparation.

DataOps Solution

Tools & processes to cope with volume, velocity, and variety of data.

Creating an ML System

- What problem to solve?
- Target deployment scenario?
- Performance objectives?

- Solution components?

- Identifying data sources
- Collecting the data

- Business ready data?
- Ingesting & storage seiection

- Identification & training
- Model evaluation
- Hyperparameter Tuning

- Model optimization (infrastructure)
- Packaging & deployment

- Performance as expected?
- Model drift detection
- Re-training required?

Inhibitors to Successful ML Implementation

Deployment & Automation: Challenges in seamless integration and automated pipelines.

Reproducibility: Difficulty in replicating model results and predictions consistently.

Diagnostics: Lack of tools for deep model performance and failure analysis.

Governance & Compliance: Meeting regulatory standards, ethics, and policy requirements.

Scalability: Issues with handling increased data volumes and user load.

Collaboration: Siloed teams (data science, engineering, business) hindering progress.

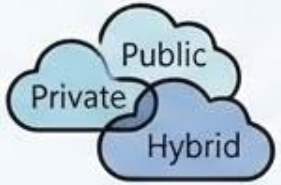
Monitoring & Management: Inadequate tracking of model drift, performance, and health over time.

Cloud Computing

Access to computing resources and storage on demand

Pay-as-you go model

Heterogeneous resources: GPUs, CPUs, storage type



Different deployment models: Public, private, hybrid cloud

Different deployment models: GPUs, CPUs, storage, hybrid cloud

Provisioning, maintenance, monitoring, life-cycle-management

Cloud and AI

AI

Harness power of Big Data and compute

Cloud

Access to Big Data

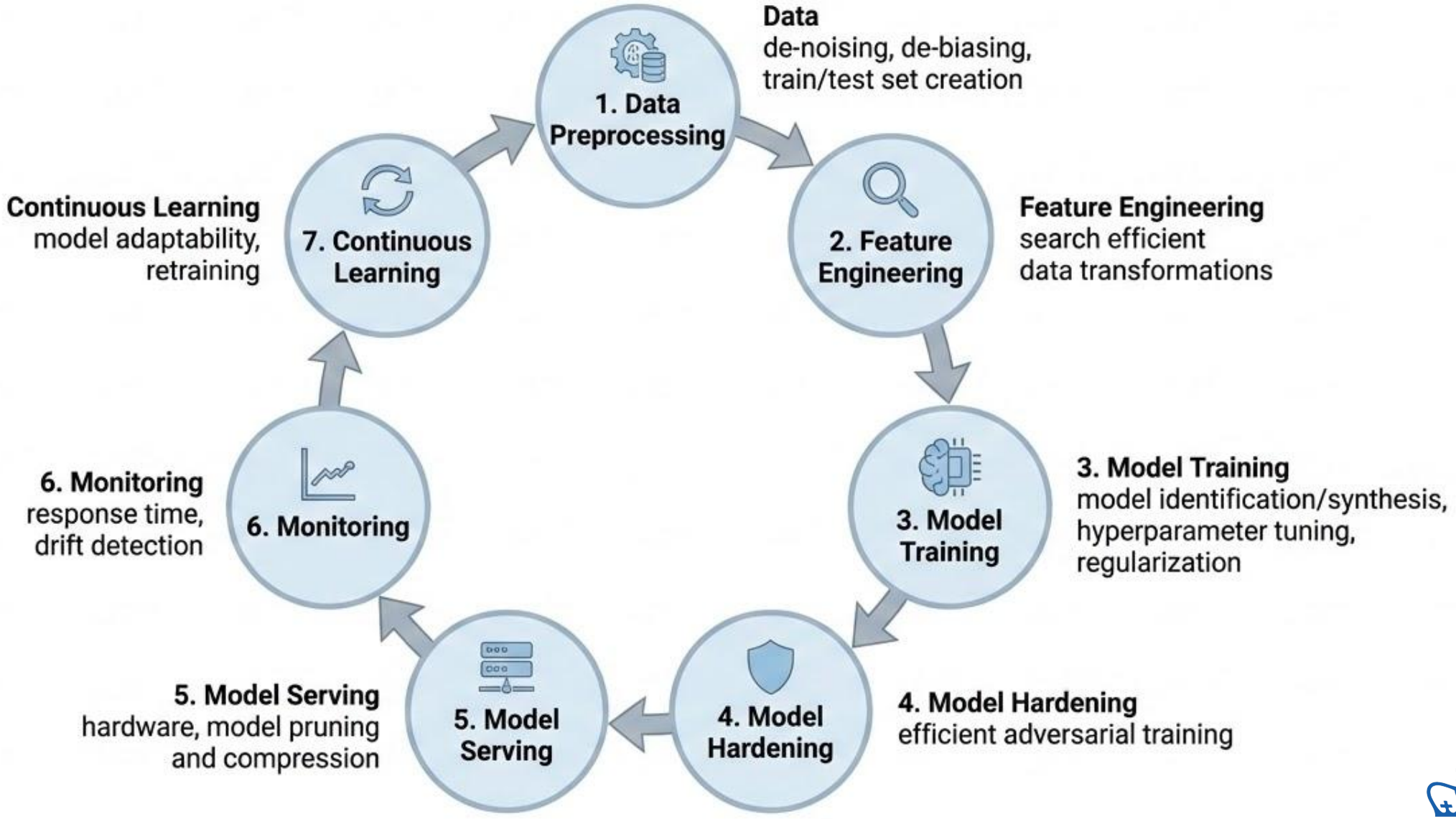
Platform to quickly develop, deploy, and test AI solutions

Ease in AI reachability

Evolution of Cloud AI: From Core ML to GenAI Services

Provider	Legacy/Core ML Service	New GenAI/LLM Service	Key Models/Features
IBM	Watson Studio	IBM watsonx.ai (https://www.ibm.com/products/watsonx)	• Granite, Llama 2, Hugging Face integration.
AWS	Amazon SageMaker	Amazon Bedrock (https://aws.amazon.com/bedrock/)	• Claude 3, Amazon Titan, Stable Diffusion.
Microsoft Azure	Azure Machine Learning	Azure AI Foundry (https://ai.azure.com/)	• GPT-4o, DALL-E 3, Phi-3.
Google	Google Cloud AI	Vertex AI (https://cloud.google.com/vertex-ai)	• Gemini 1.5 Pro/Flash, Imagen.

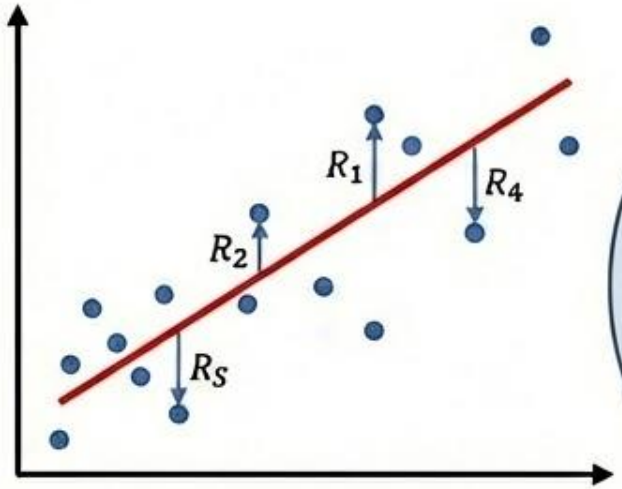
AI Model Training Lifecycle



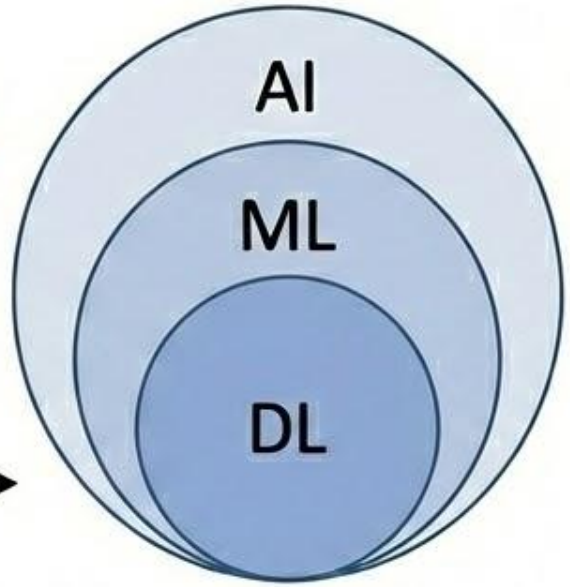
Class 1: Deep Learning (DL) Fundamentals

From Machine Learning to Deep Learning

Traditional Machine Learning

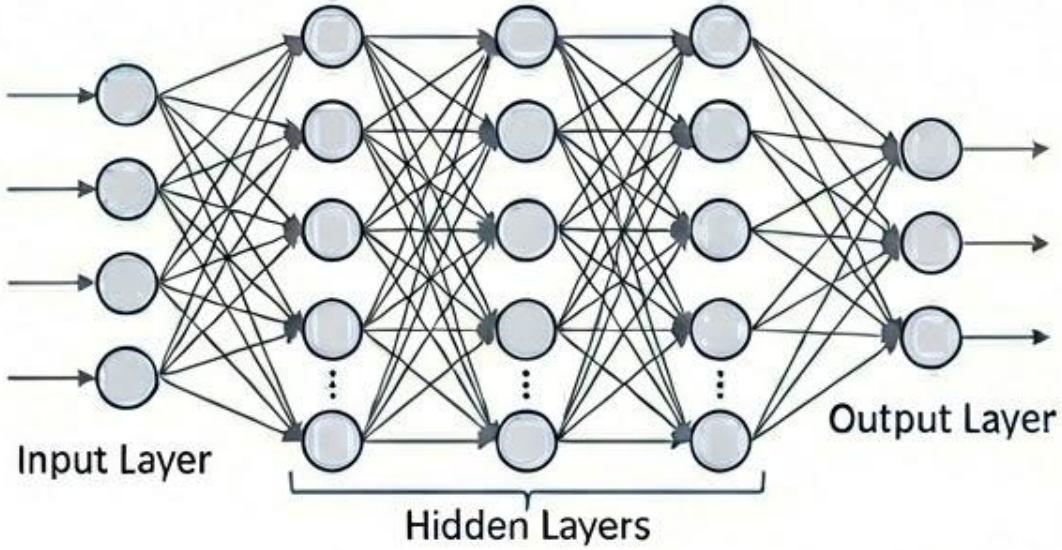


$$Y = f(X) + \epsilon$$



- **ML Core:** Learning from data without explicit programming (e.g., minimized Error).
- **Linear Regression Metrics:**
  - **RSS (Residual Sum of Squares)**: Amount of variability not explained by the model. $\text{RSS} = \sum(y_i - \hat{y}_i)^2$
  - **TSS (Total Sum of Squares)**: Total variability inherent in the model. $\text{TSS} = \sum(y_i - \bar{y})^2$
  - **R^2 (R-squared)**: Proportion of variability in Y that can be explained using X. $R^2 = 1 - (\text{RSS} / \text{TSS})$

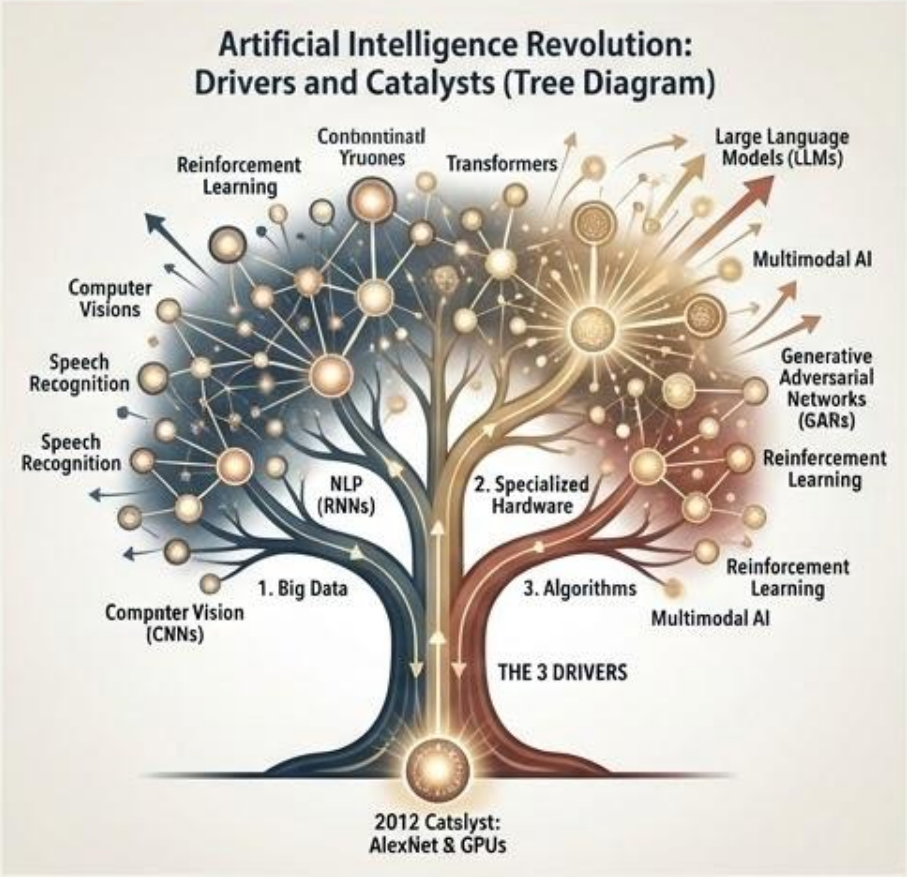
Deep Learning



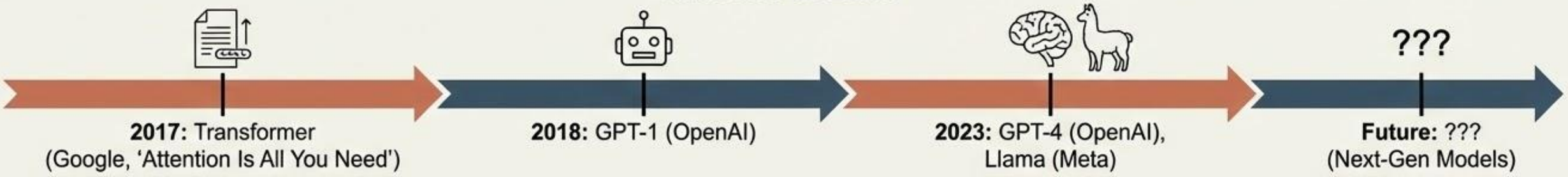
- **DL Core:** Stacking "hidden layers" to capture non-linear, complex patterns.
- The **Learning Loop**:
  - **Forward Phase**: Calculate activations & output.
  - **Backward Phase**: Update weights to decrease loss (Backpropagation).

The “Cambrian Explosion” of AI

- **The Spark** (2012): AlexNet (GPU-based model) proved DL works at scale.
- **The 3 Drivers:**
  1. Big Data (The fuel).
  2. Specialized Hardware (GPUs/TPUs).
  3. Algorithms (Distributed Training / Transformers).
- **The Result:** A rapid evolution from simple classifiers to Large Language Models (LLMs).

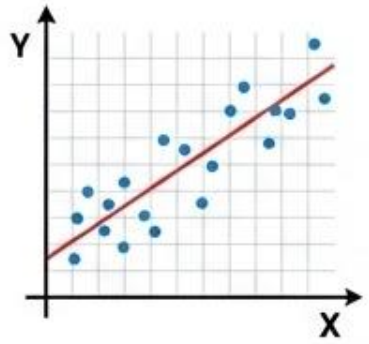


Timeline of LLMs



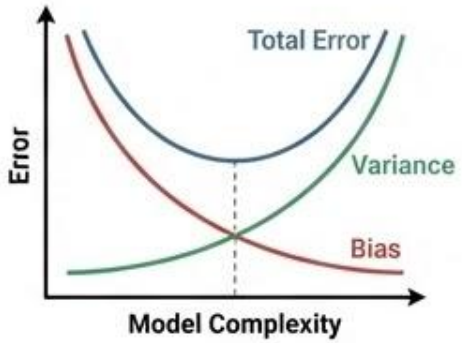
Core Concepts of Traditional Machine Learning

Model Fitting & Error (Regression)



- ⚙️ **Linear Regression** dynamics ($Y=f(X)+\epsilon$).
- 📊 **Quantifying Error:** RSS (Residual Sum of Squares) vs. TSS (Total Sum of Squares).
- ⚙️ **Goodness of Fit:** R^2 and MSE (Mean Squared Error).

Generalization & Trade-offs



- ⚙️ **Overfitting** vs. **Underfitting** (The Generalization Gap).
- 📊 The **Bias-Variance Trade-off:** Balancing approximation error vs. estimation error.
- ⚙️ **Regularization:** Using Lambda (λ), L1 (Lasso), and L2 (Ridge) to control complexity.

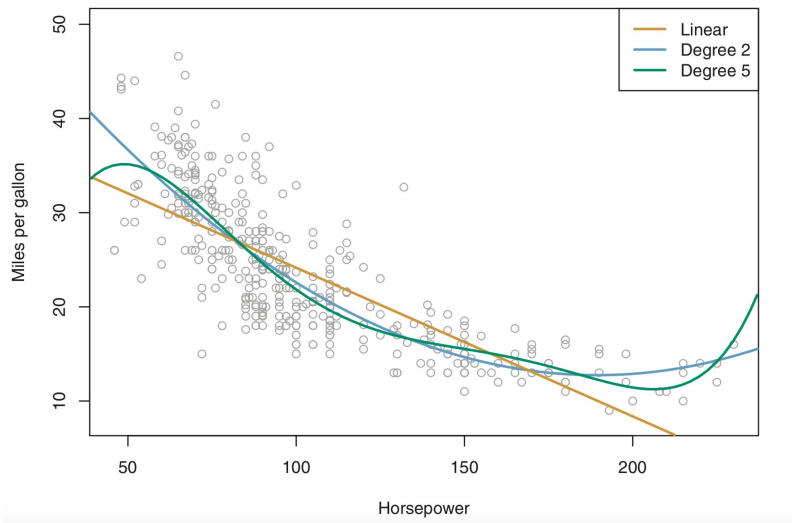
Performance Evaluation (Classification)

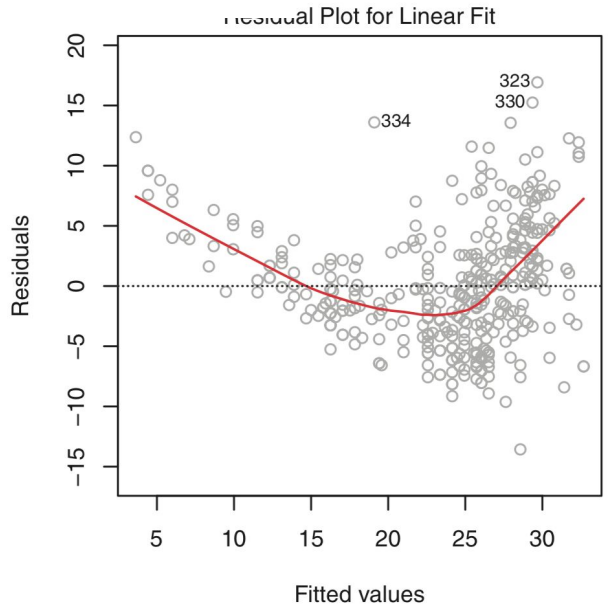
True Positive (TP)	False Positive (FP)
False Negative (FN)	True Negative (TN)

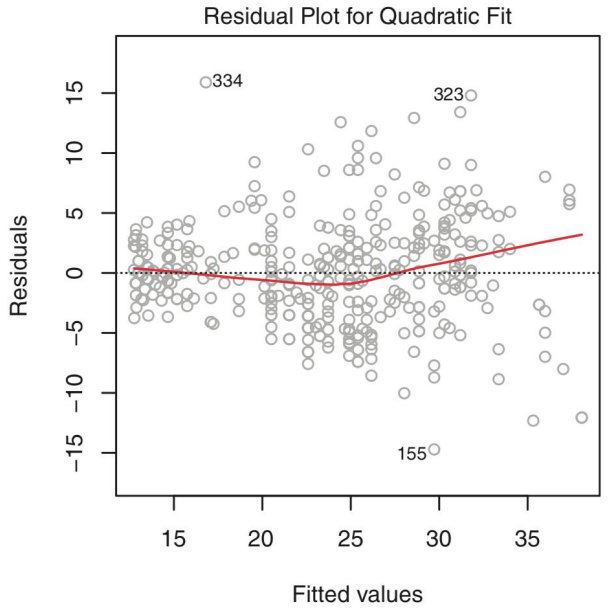
- 🖱️ **Confusion Matrix:** TP, TN, FP, FN.
- 📈 **Metrics:** Accuracy, Precision, Recall (Sensitivity), and Specificity.
- ✅ **Balancing:** The F1-Score and the trade-off between Precision and Recall.

Linear Regression

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \cdots + \hat{\beta}_p x_p$$







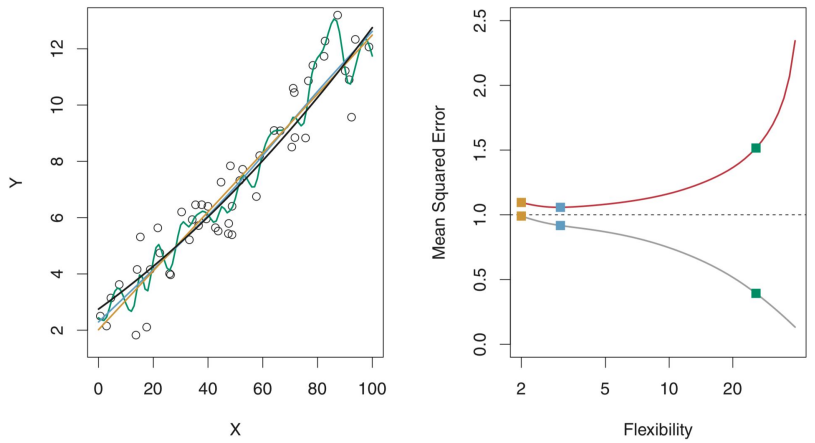
$$\text{TSS} = \sum (y_i - \bar{y})^2$$

$$\text{RSS} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

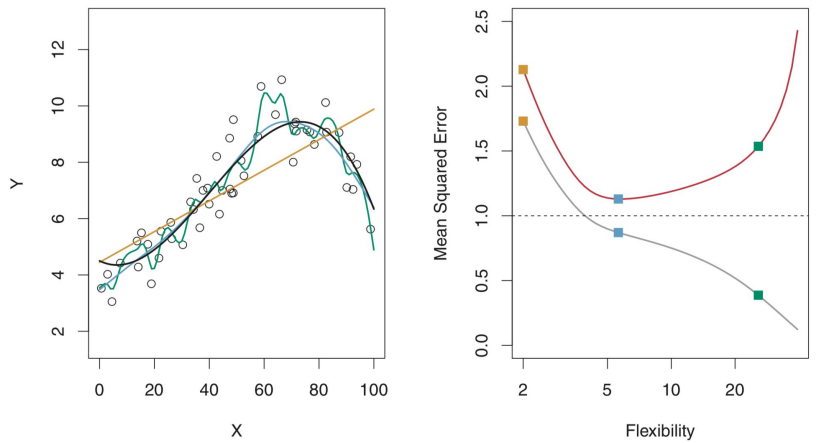
$$R^2 = \frac{\text{TSS} - \text{RSS}}{\text{TSS}} = 1 - \frac{\text{RSS}}{\text{TSS}}$$

Mean Square Error (MSE)

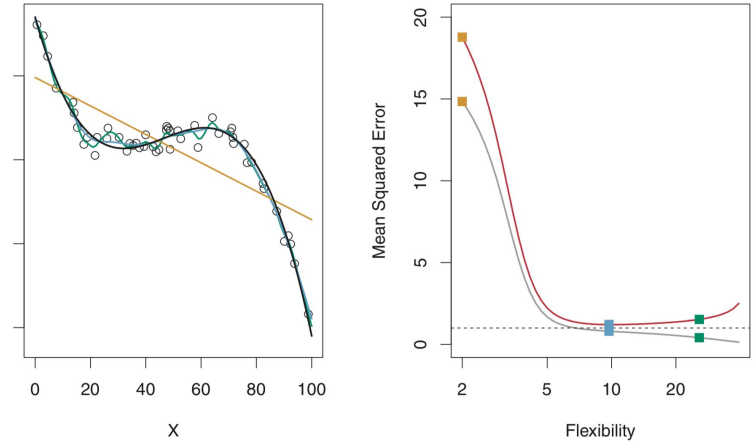
CASE 1



CASE 2



CASE 3



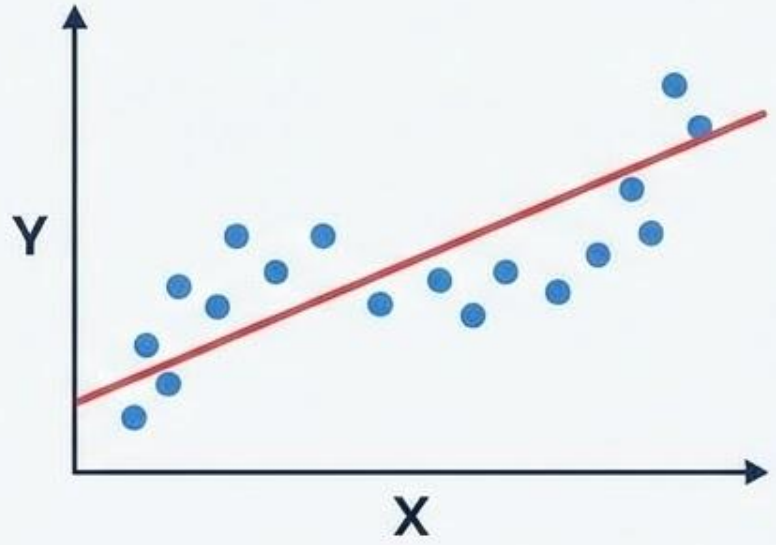
true value $Y = f(X) + \epsilon.$

predicted $\hat{Y} = \hat{f}(X)$

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{f}(x_i))^2$$

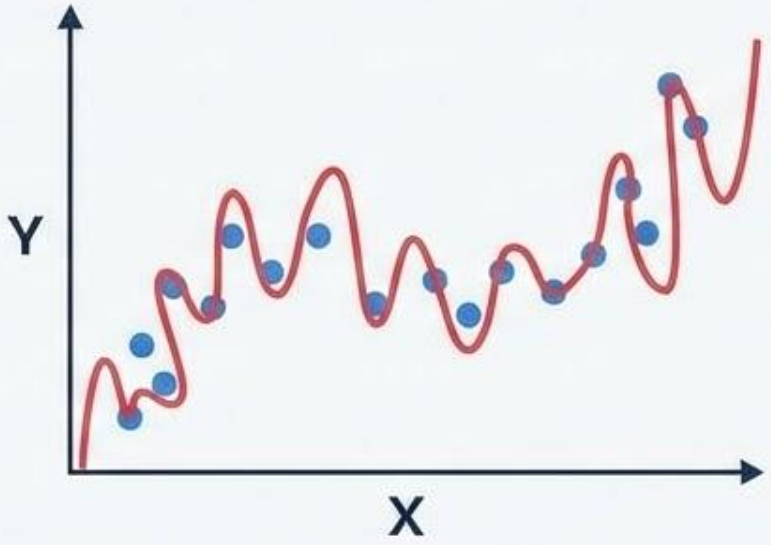
Overfitting and Underfitting

UNDERFITTING



- Model is **not complex enough** to **capture pattern** in the training data.
- Suffers from low performance on both training and unseen data.

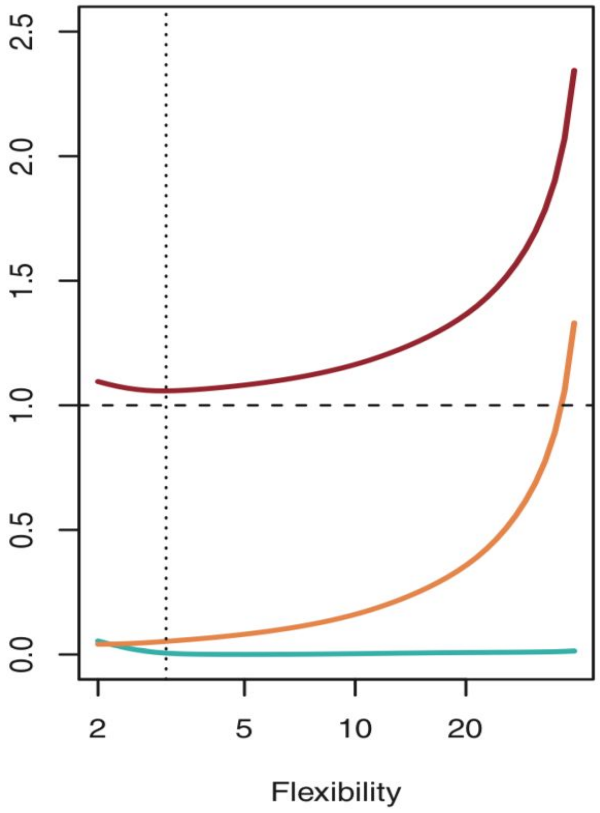
OVERFITTING



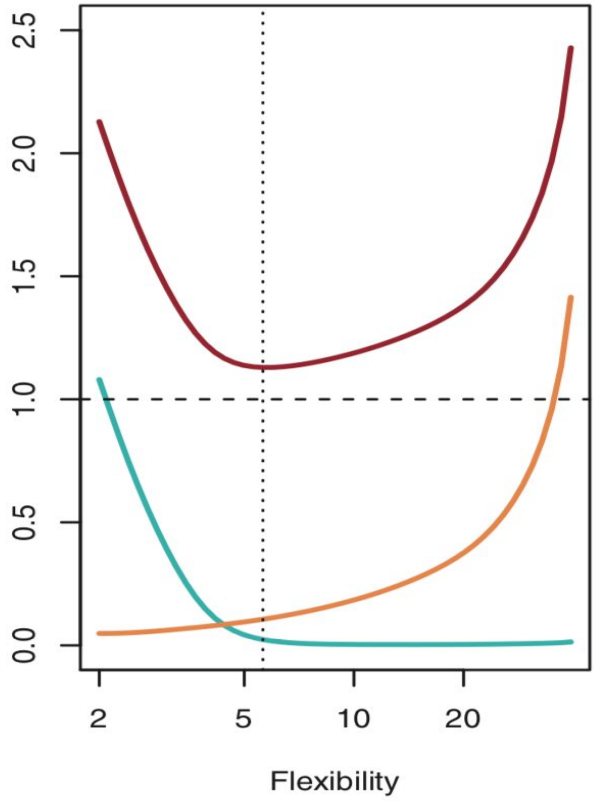
- Model performs **extremely well** on **training** data.
- Does **not generalize** well to unseen data (test data).

Model Complexity Tradeofffs

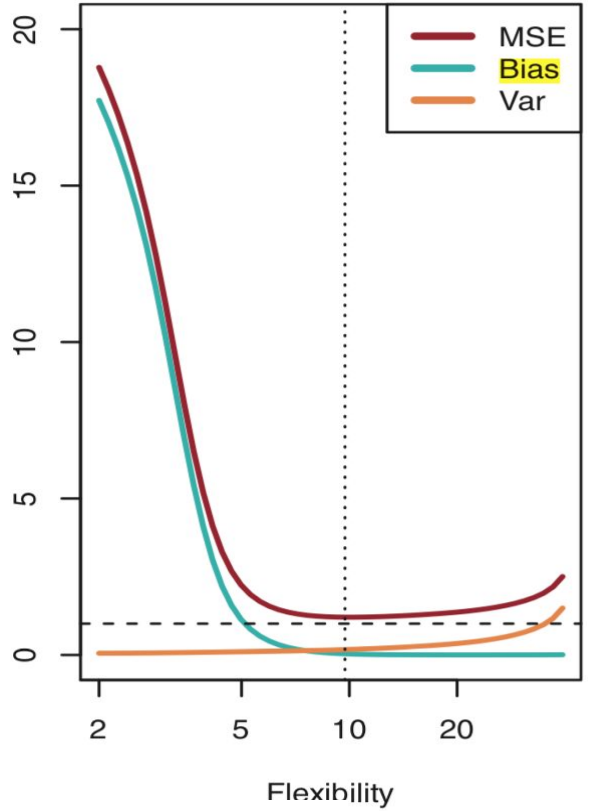
CASE 1



CASE 2

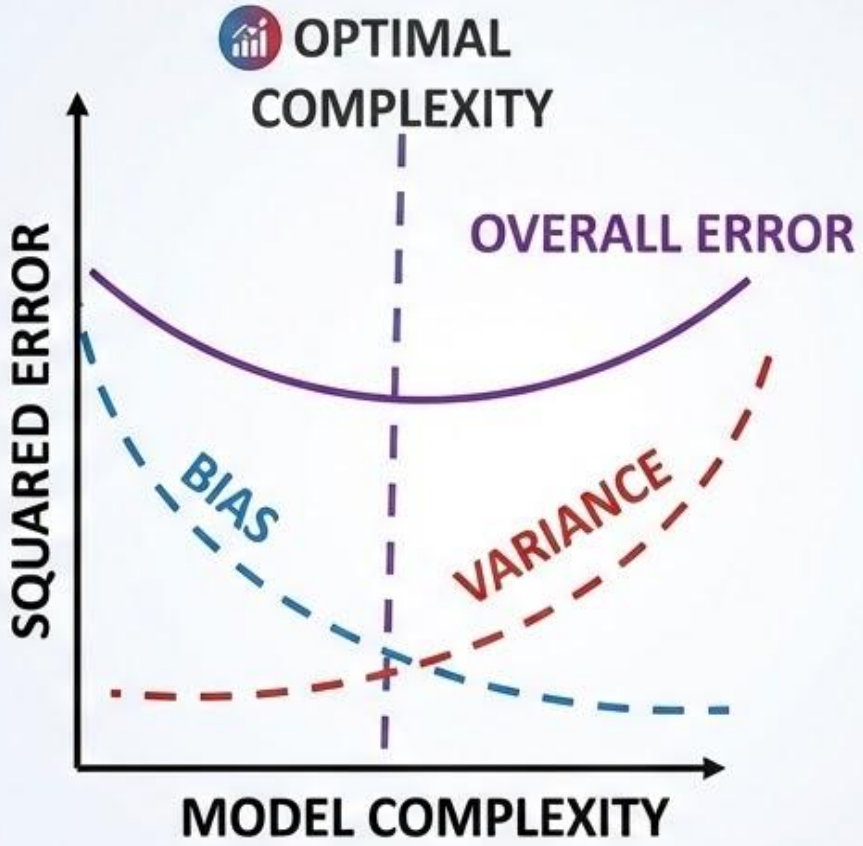


CASE 3



$$E\left(y_0 - \hat{f}(x_0)\right)^2 = \mathrm{Var}(\hat{f}(x_0)) + [\mathrm{Bias}(\hat{f}(x_0))]^2 + \mathrm{Var}(\epsilon)$$

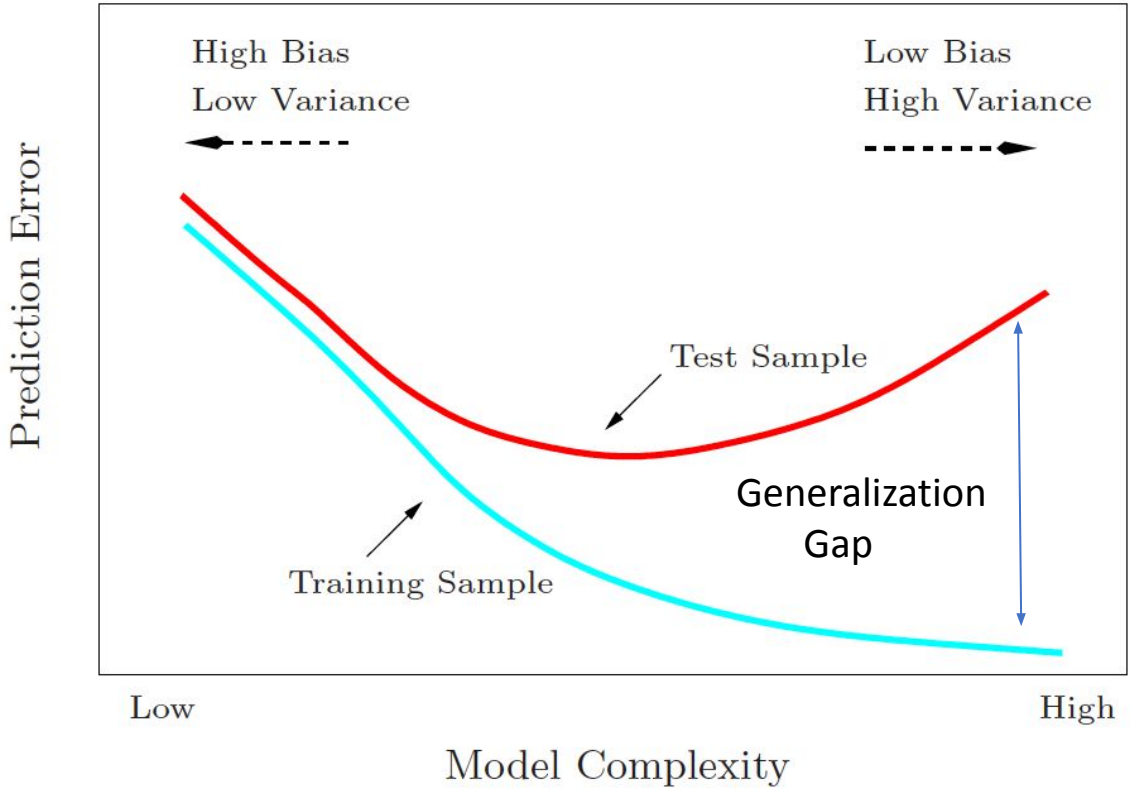
The Bias-Variance Trade-Off



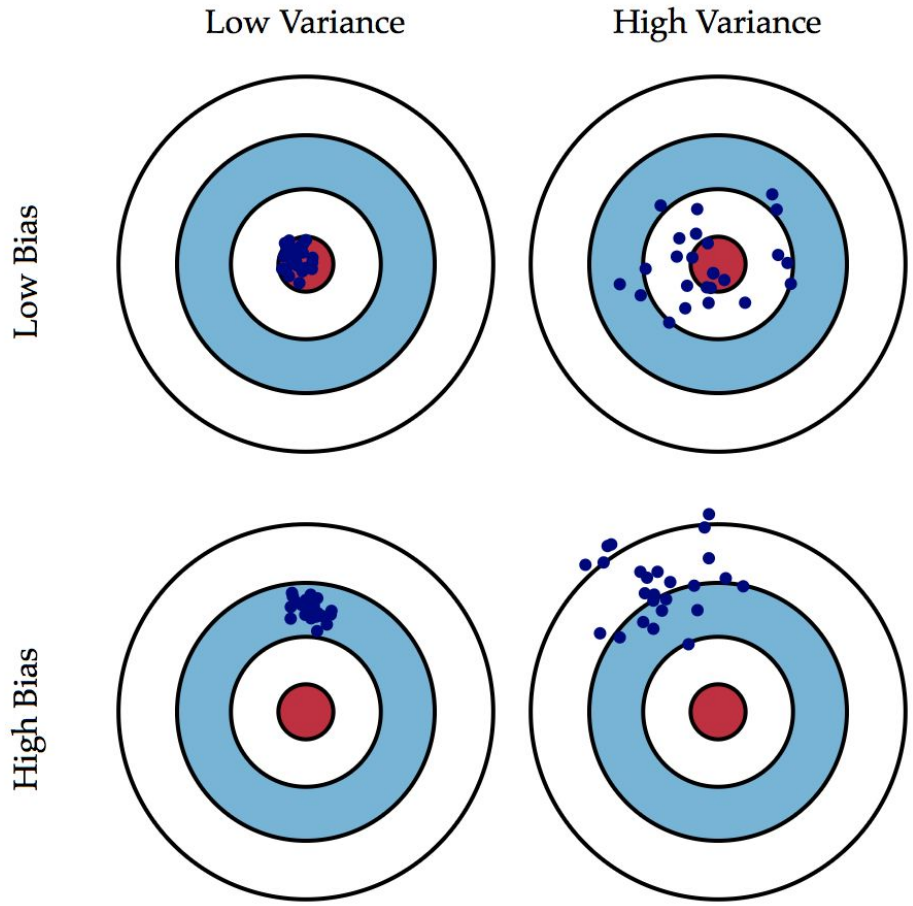
Optimal point of model complexity is somewhere in the middle.

Model Complexity Tradeoffs

- Simple model
  - Fail to completely capture the relationship between features
  - Introduces bias: Consistent test error across different choices of training data
  - Low variance
  - Increasing training data does not help in reducing bias
- Complex model captures nuances in training data causing Overfitting
  - Low bias
  - Train error << Test error
  - With different training instances, the model prediction for same test instance will be very different – High Variance



Bias-Variance Tradeoff



High Bias

Increase model complexity

Low Bias

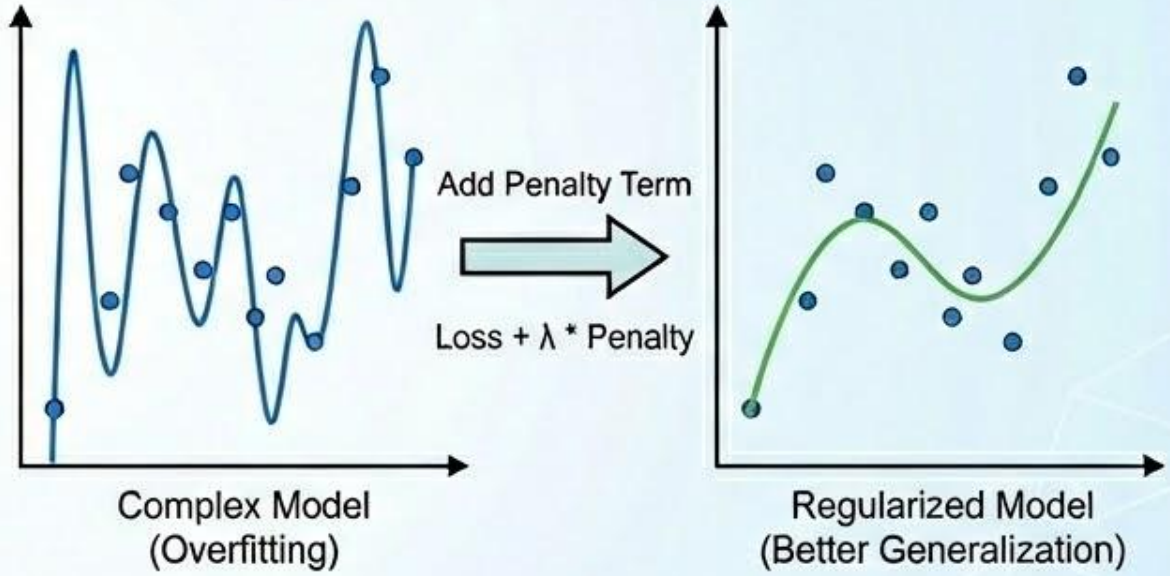
High Variance

Increase training data

Low Variance

Regularization

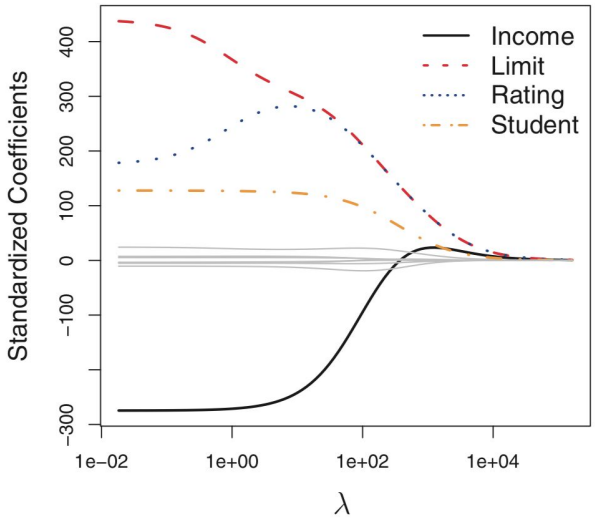
- Techniques used to improve generalization of a model by reducing its complexity
- Techniques to make a model perform well on test data often at expense of its performance on training data
- Avoid overfitting, reduce variance
- Simpler models are preferable: low memory, increase interpretability
- However simpler models may reduce the expressive power of models
- Sample techniques:
  - Parameter norm penalties
    - L_2 and L_1 norm weight decay
  - Noise injection
    - Dropout

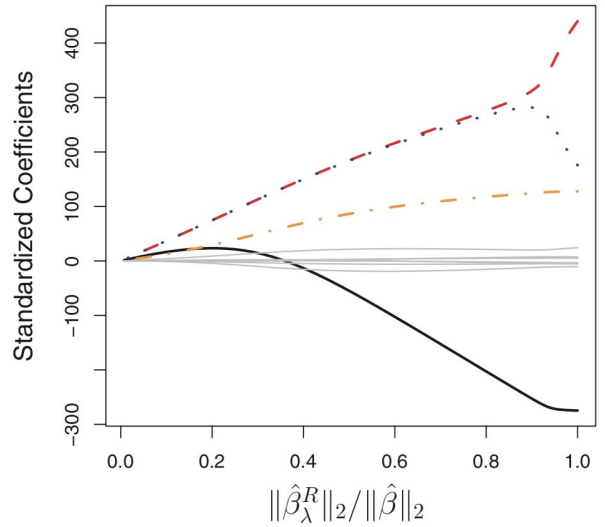


Regularization in Regression

L_2 Regularization Loss (Ridge Regression)

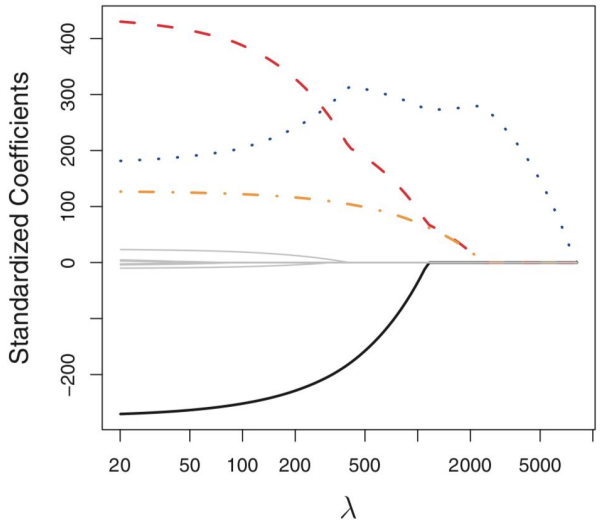
$$\sum_{i=1}^{n}\left(y_i-\beta_0-\sum_{j=1}^{p}\beta_j x_{ij}\right)^2+\lambda\sum_{j=1}^{p}\beta_j^2$$

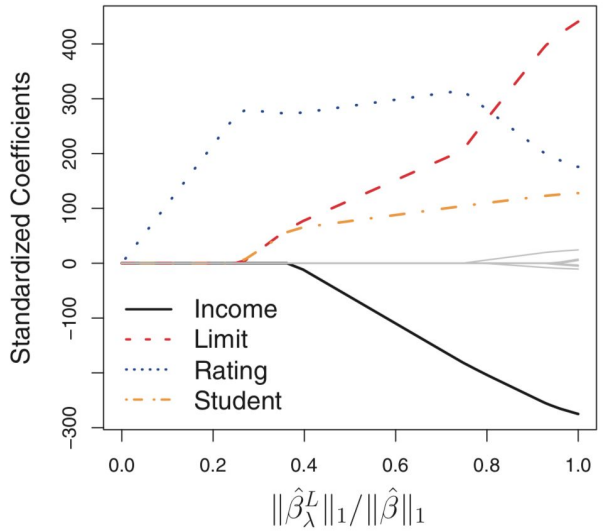




L_1 Regularization Loss (LASSO)

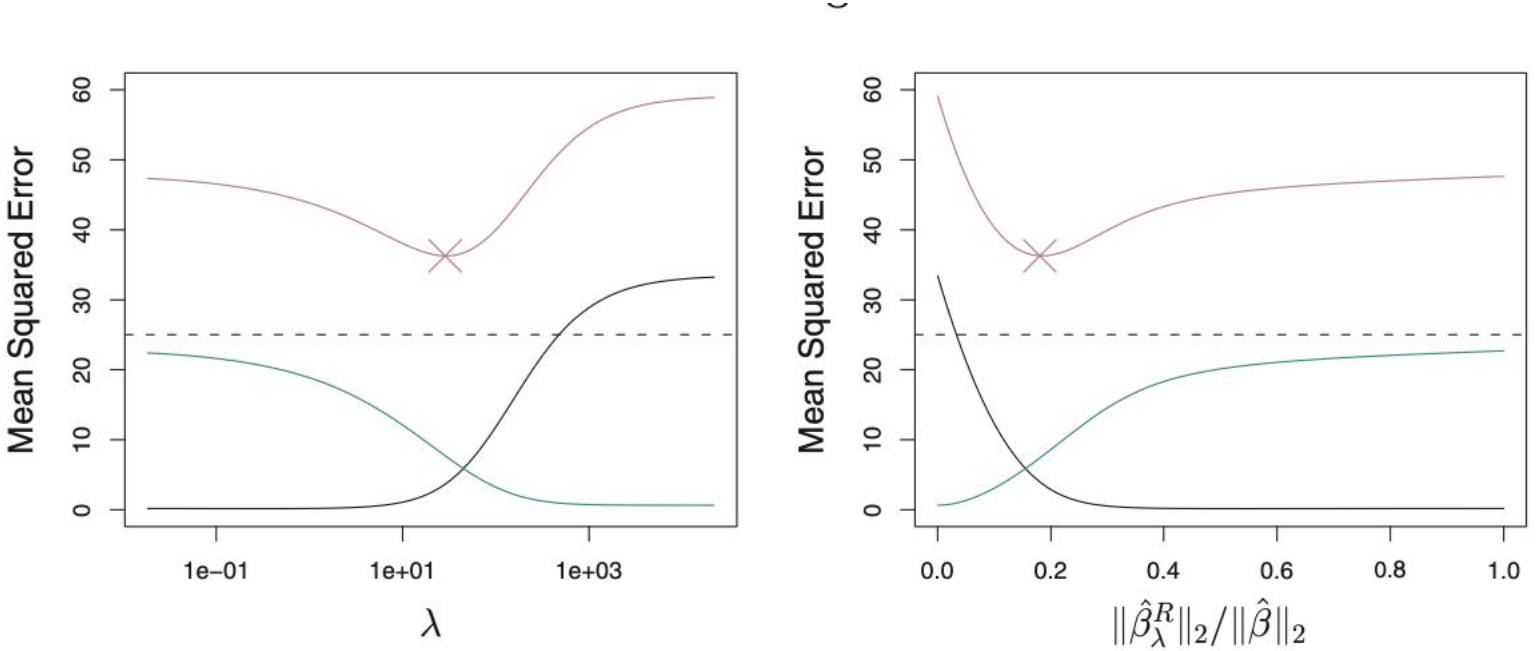
$$\sum_{i=1}^{n}\left(y_i-\beta_0-\sum_{j=1}^{p}\beta_j x_{ij}\right)^2+\lambda\sum_{j=1}^{p}|\beta_j|$$





What value of lambda to choose ?

Bias-variance tradeoff with Lambda



Performance Metrics

Algorithmic performance

Accuracy

Precision

Recall

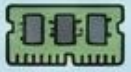
F1-score

System performance

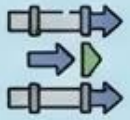
Training time

Inference latency

Training cost

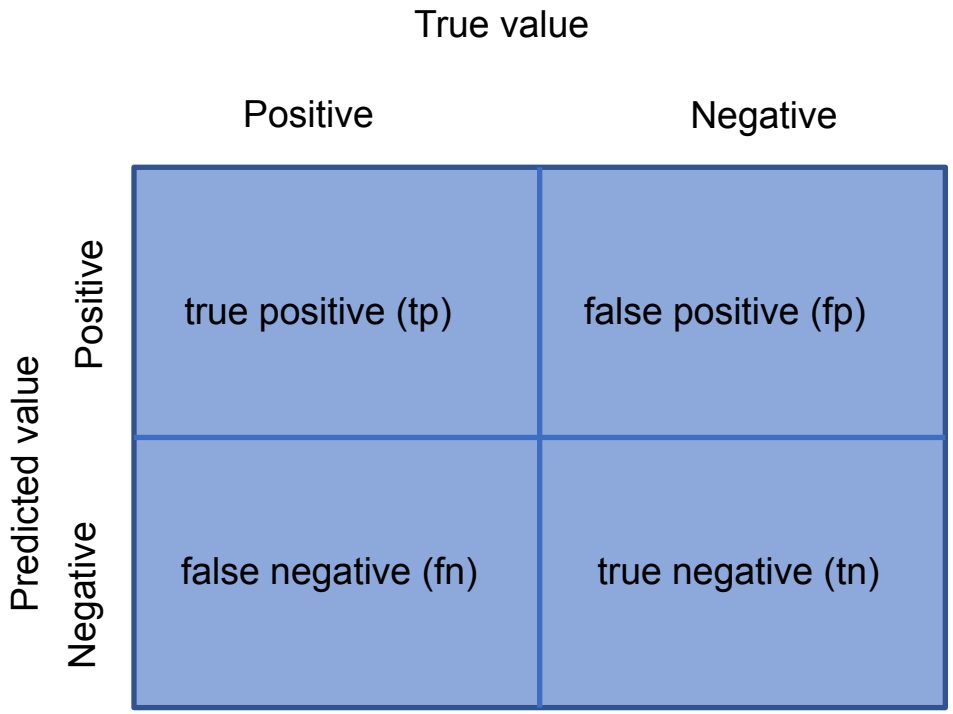


Memory requirement



Training throughput

Accuracy, Precision, Recall, Specificity



$$\text{Accuracy} = \frac{tp + tn}{tp + tn + fp + fn}$$

$$\text{Precision} = \frac{tp}{tp + fp}$$

False discovery rate = 1-Precision

$$\text{Recall} = \frac{tp}{tp + fn}$$

$$\text{True negative rate} = \frac{tn}{tn + fp}$$

Sensitivity, True positive rate

Specificity

False positive rate = 1-Specificity

Balanced accuracy = $(Sensitivity + Specificity)/2$

Considers all entries in the confusion matrix
Value lies between 0 (worst classifier) and 1 (best classifier)

Balancing Precision and Recall

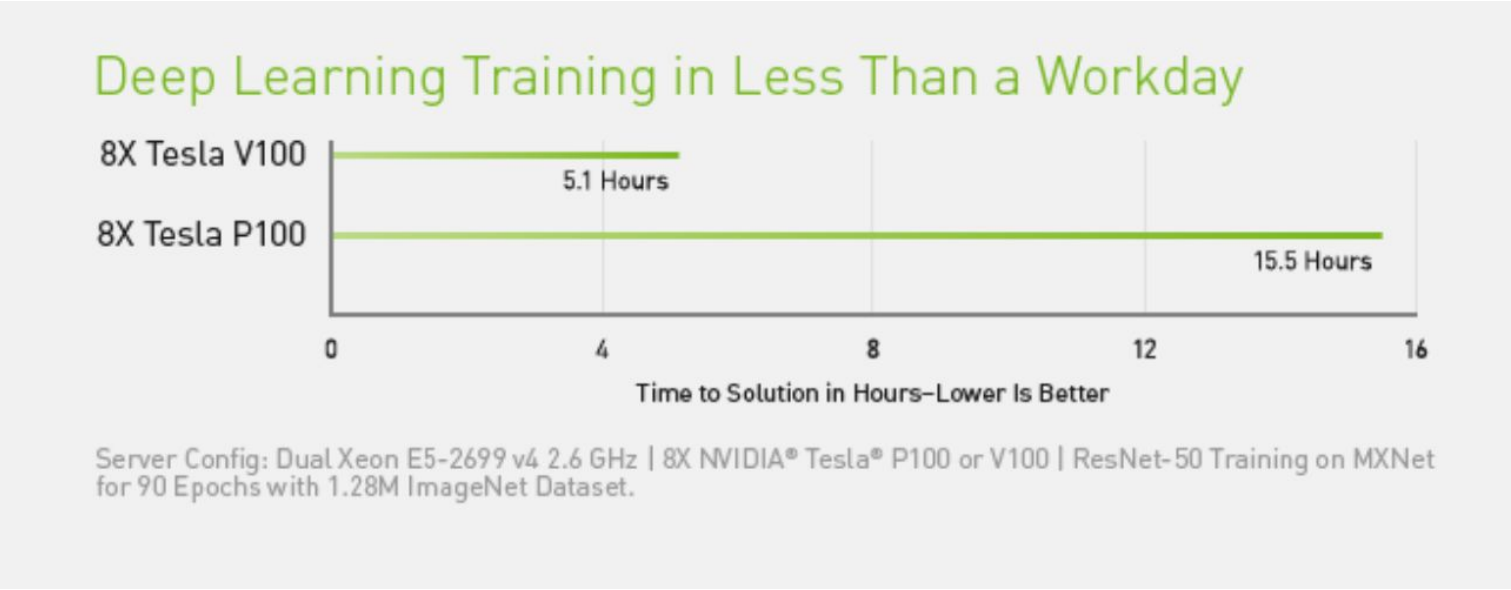
- $$F_1 = \frac{2}{\mathrm{recall}^{-1} + \mathrm{precision}^{-1}} = 2 \cdot \frac{\mathrm{precision} \cdot \mathrm{recall}}{\mathrm{precision} + \mathrm{recall}} = \frac{\mathrm{tp}}{\mathrm{tp} + \frac{1}{2}(\mathrm{fp} + \mathrm{fn})}.$$

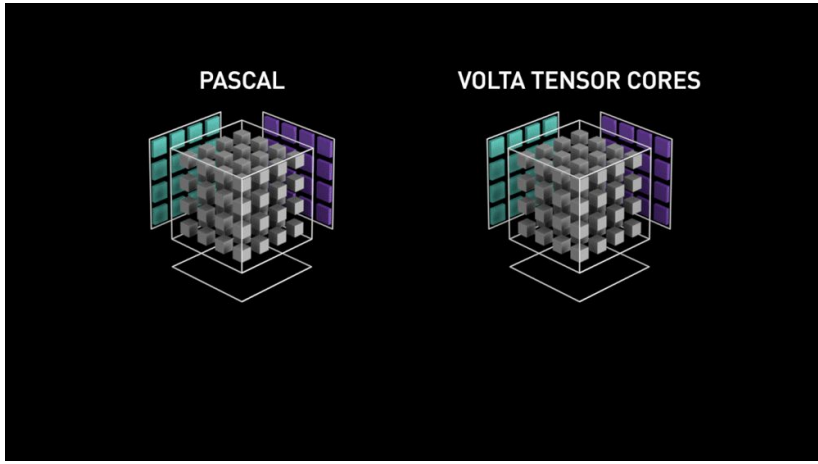
F_β

$$F_\beta = (1 + \beta^2) \cdot \frac{\mathrm{precision} \cdot \mathrm{recall}}{\beta^2 \cdot \mathrm{precision} + \mathrm{recall}}$$

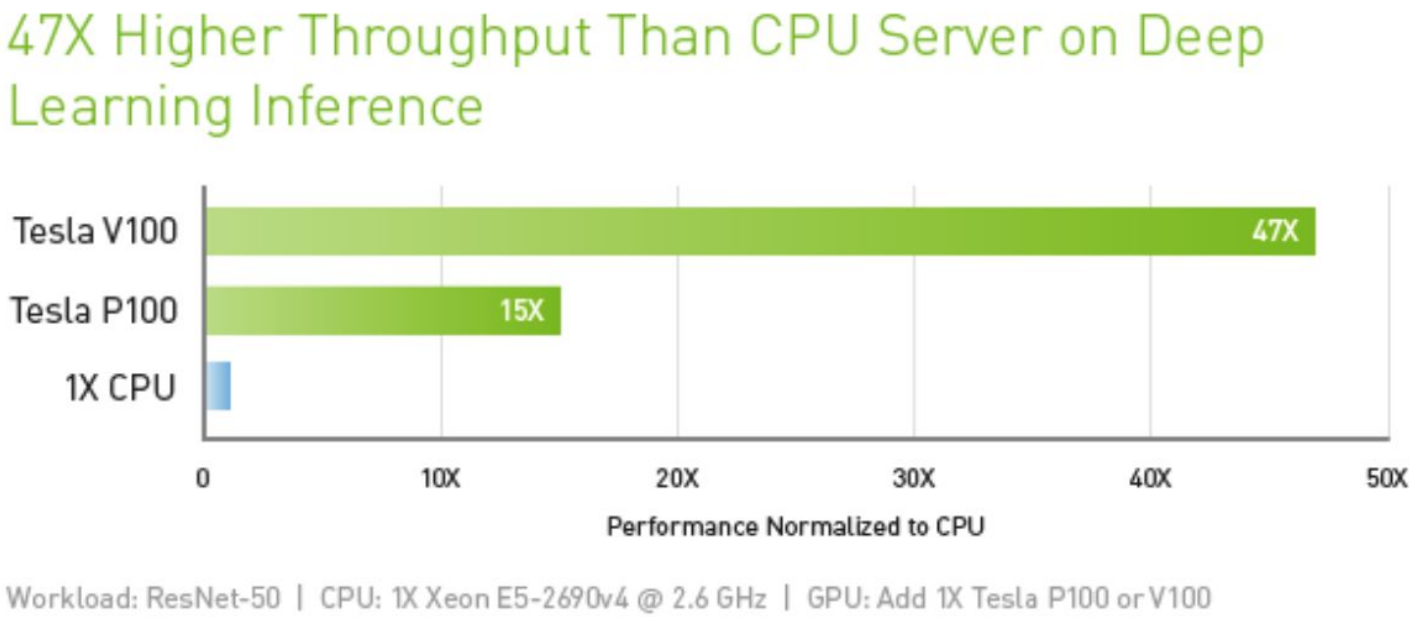
DL Training Time

- DL performance is closely tied to the hardware
  - compute power, memory, network
  - Tesla V100: 640 tensor cores (> 100 TFLOPS), 16 GB
  - NVIDIA NVLink: 300 GB/s
  - Volta optimized CUDA libraries

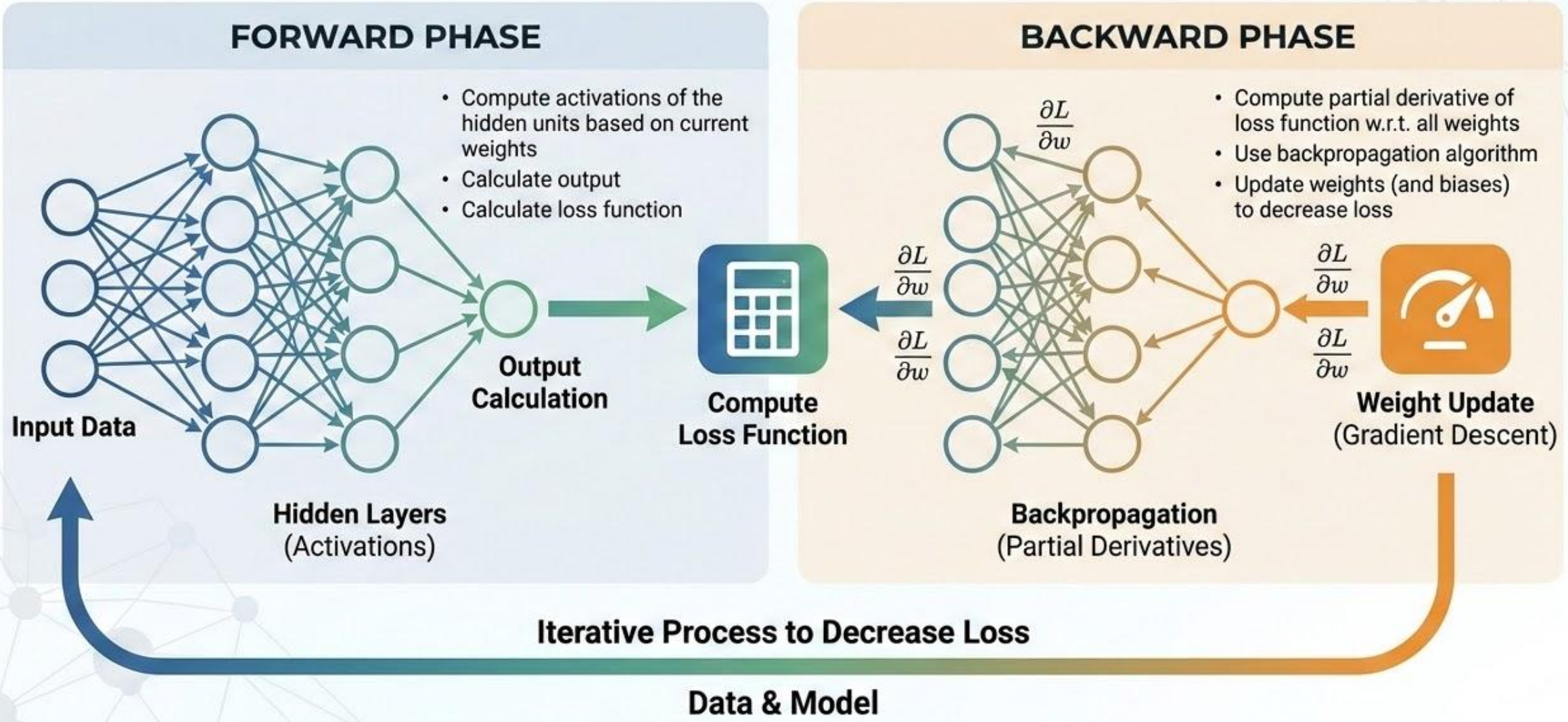




DL Inference Throughput



DEEP LEARNING TRAINING STEPS



Gradient Descent

$$L = \sum_{i=1}^{n} L_i$$

$$\overline{W} \Leftarrow \overline{W} - \alpha \frac{\partial L}{\partial \overline{W}}$$

- Loss is calculated over all the training points at each weight update
- Memory requirements may be prohibitive

Stochastic Gradient Descent (SGD)

$$\overline{W} \Leftarrow \overline{W} - \alpha \frac{\partial L_i}{\partial \overline{W}}$$

- Loss is calculated using one training data at each weight update
- Stochastic gradient descent is only a randomized approximation of the true loss function.

Mini-batch Gradient Descent

•

$$\overline{W} \Leftarrow \overline{W} - \alpha \sum_{i \in B} \frac{\partial L_i}{\partial \overline{\overline{W}}}$$

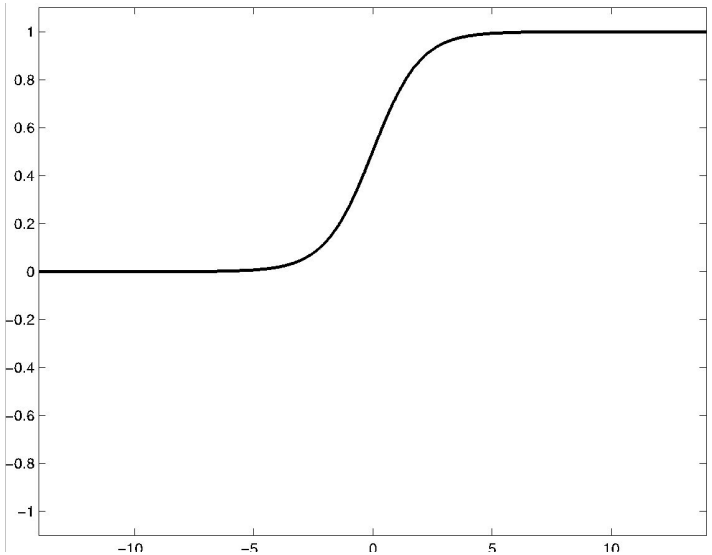
- A batch B of training points is used in a single update of weights
- Increases the memory requirements. Layer outputs are matrices instead of vectors. In backward phase, matrices of gradients are calculated.

Hyperparameters in Deep Learning

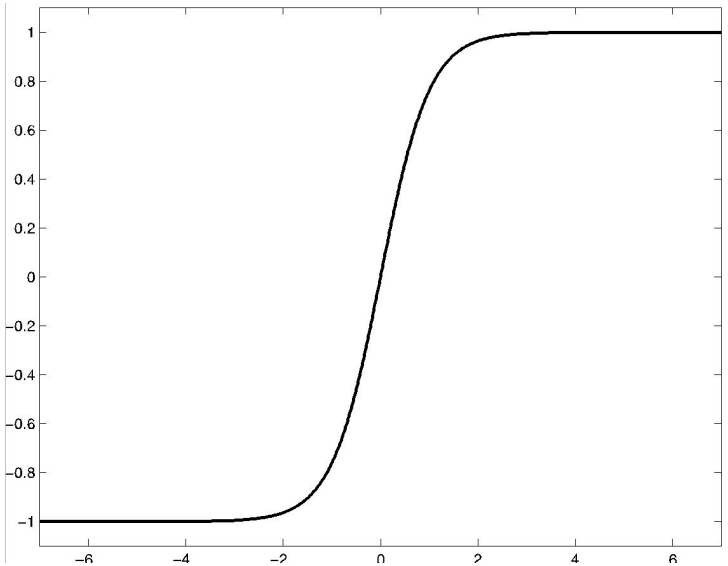
- Network architecture: number of hidden layers, number of hidden units per later
- Activation functions
- Weight initializer
- Learning rate
- Batch size
- Momentum
- Optimizer

Popular Activation Functions

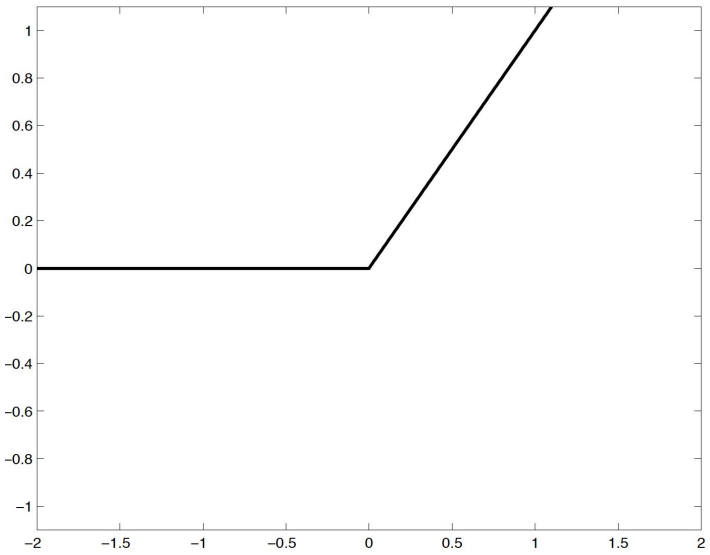
Sigmoid $\phi(z) = \frac{1}{1+e^{-z}}$



Tanh $\phi(z) = \frac{e^{2z}-1}{e^{2z}+1}$

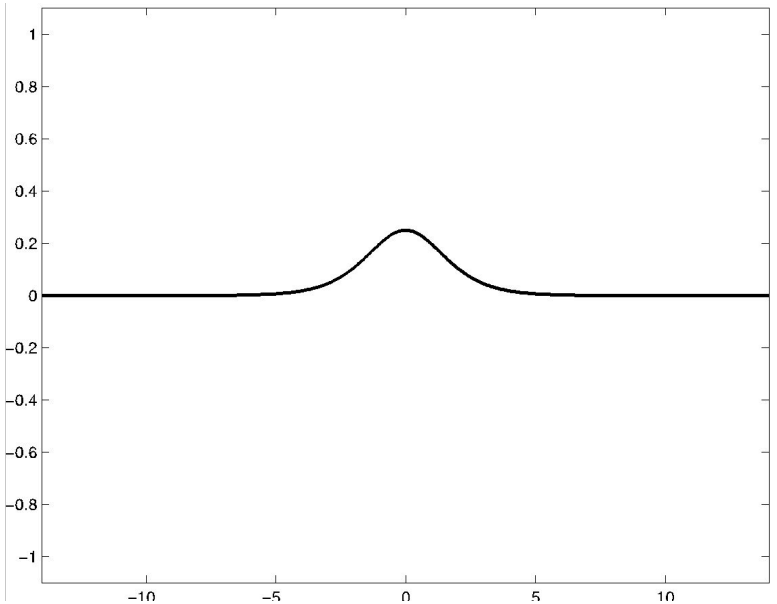


ReLU (Rectified Linear Unit)
$\phi(z) = \max\{z, 0\}$

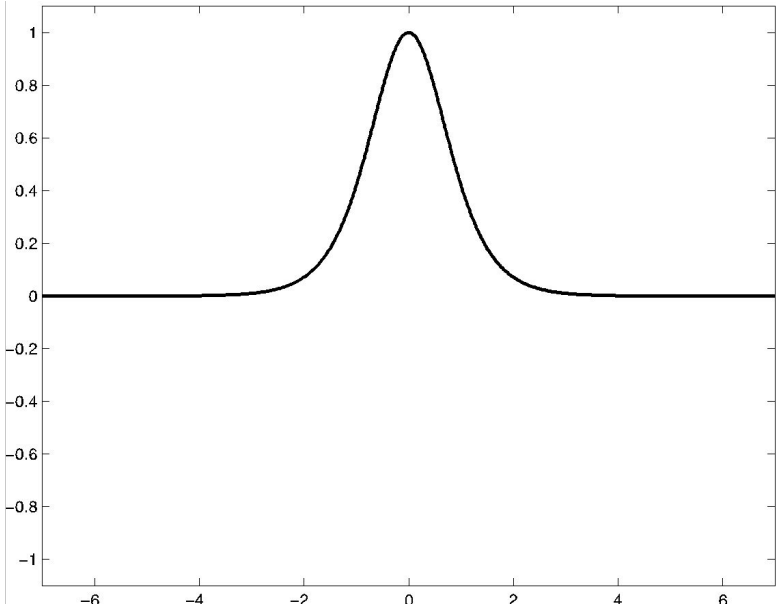


Activation Functions Derivatives

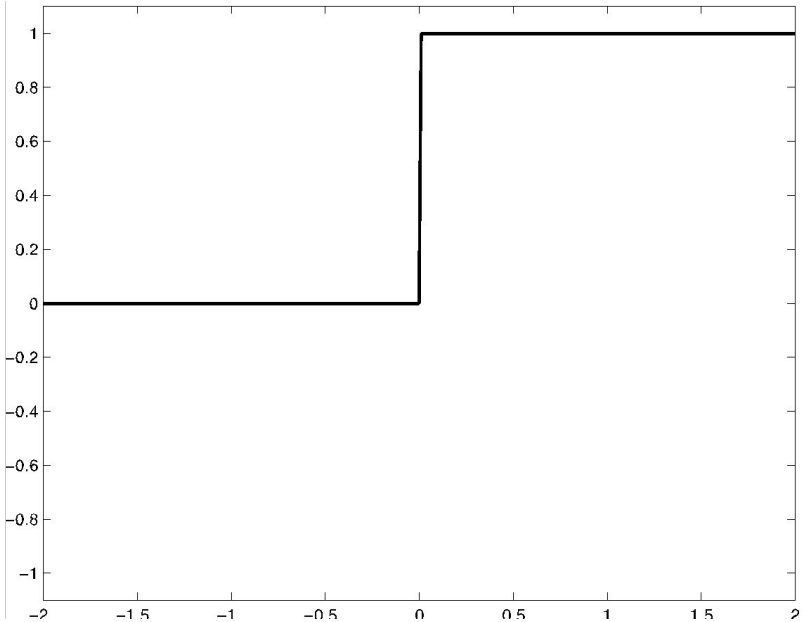
sigmoid



tanh



ReLU

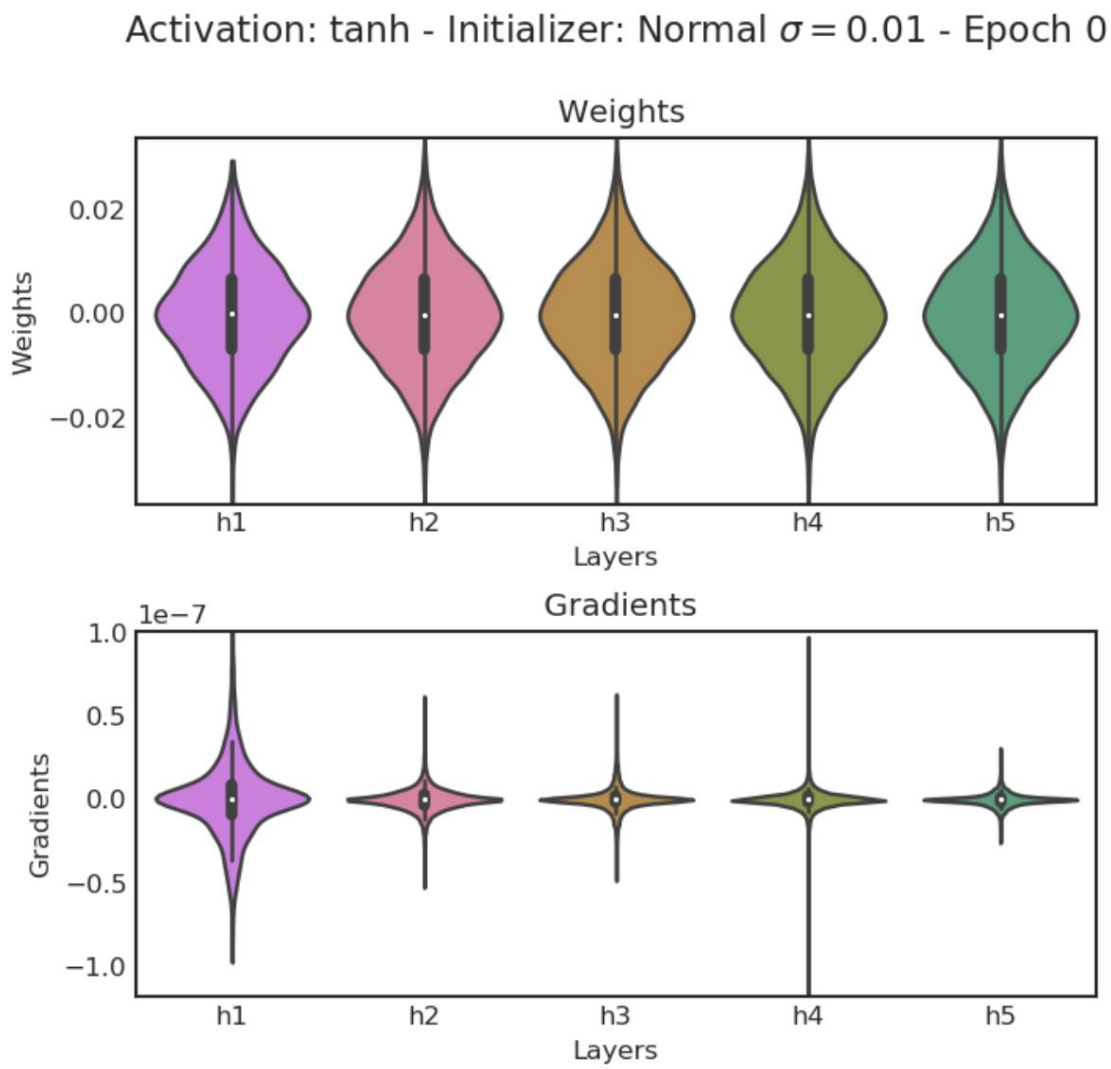


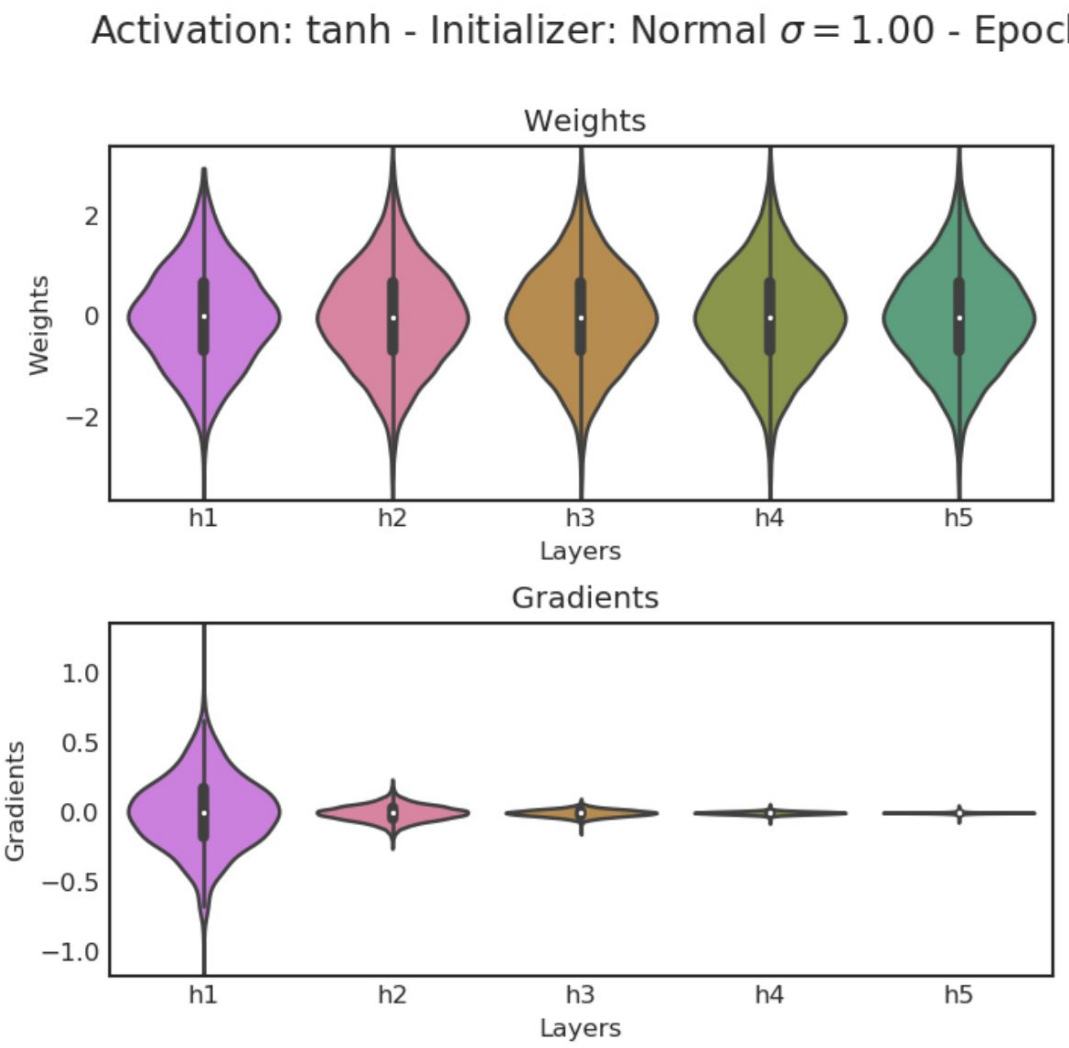
- Sigmoid and tanh derivatives vs ReLU
- Sigmoid and tanh gradients saturate at large values of argument; very susceptible to vanishing gradient problem
- ReLU is faster to train; most commonly used activation function in deep learning

Weight Initialization

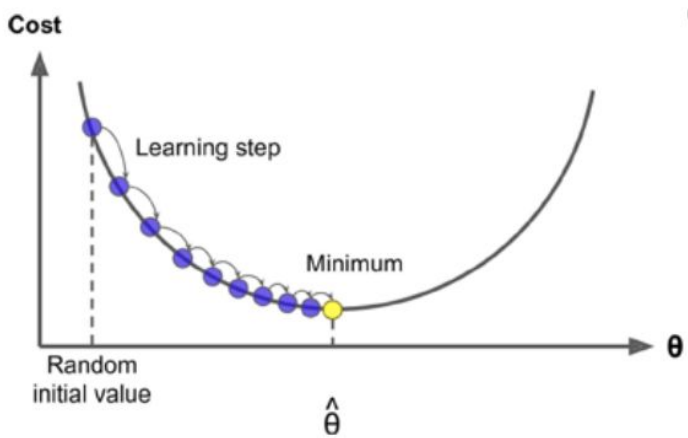
- Initializing all weights to same value will cause neurons to evolve symmetrically
- Generally, biases are initialized with 0 values and weights with random numbers; Initializing weights to random values breaks symmetry and enables different neurons to learn different features
- Initial value of weights is important.
  - Poor initializations can lead to bad convergence or no learning.
  - Instability across different layers (vanishing and exploding gradients).

Vanishing and Exploding Gradients

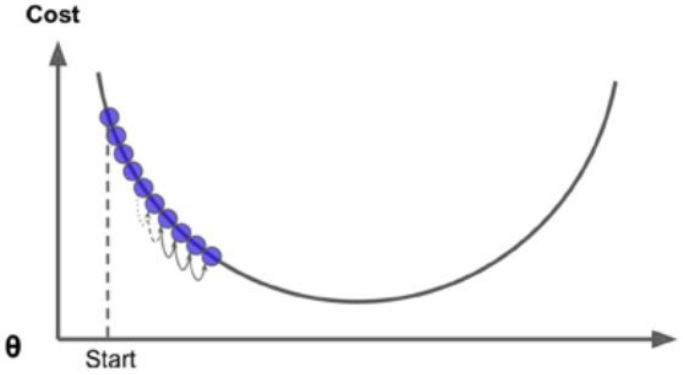




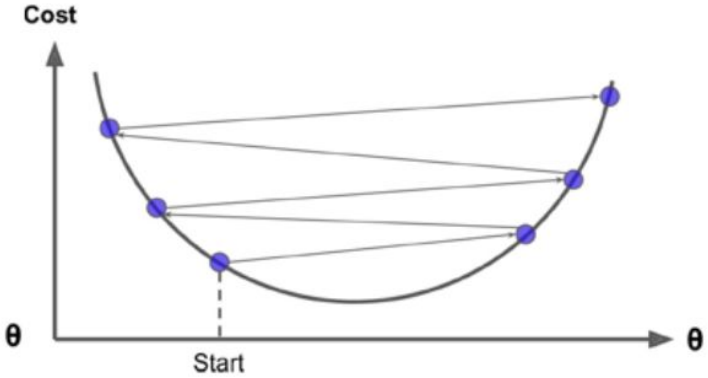
Learning rate: large value vs small value



Optimum learning rate : The model adjusts weights (θ) in subsequent training loops to arrive at cost minima.



Slow learning rate : Converges to cost minima but very slowly.



*Fast learning rate : **may** not converge to cost minima and the cost might keep increasing with further training loops.*

Image Credit : "Hands-on Machine Learning with Scikit-Learn and TensorFlow " by Aurelien Geron

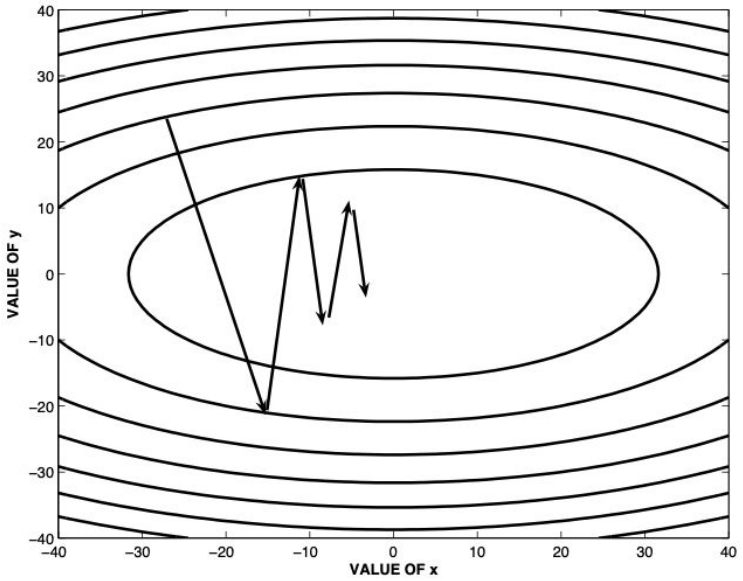
Learning rate schedule

- Start with a higher learning rate to explore the loss space => find a good starting values for the weights
- Use smaller learning rates in later steps to converge to a minima =>tune the weights slowly
- Different choices of decay functions:
  - exponential, inverse, multi-step, polynomial
  - babysitting the learning rate
- Training with different learning rate decay
  - Keras learning rate schedules and decay
- Other new forms: cosine decay

Batch size

- Effect of batch size on learning
- Batch size is restricted by the GPU memory and the model size
  - Model and batch of data needs to remain in GPU memory for one iteration
- Are you restricted to work with small size mini-batches for large models and/or GPUs with limited memory
  - No, you can simulate large batch size by delaying gradient/weight updates to happen every n iterations (instead of n=1) ; supported by frameworks

Gradient Descent Convergence



(b) Loss function is elliptical bowl

$$L = x^2 + 4y^2$$

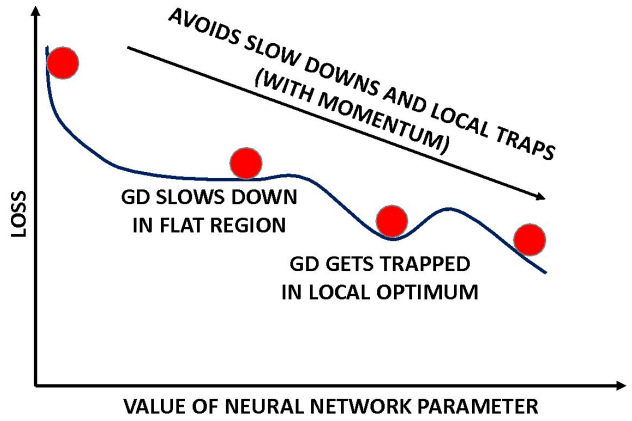
- GD convergence is poor due to difference in gradient values along different dimensions
- Effective descent direction gets away from the minima if we use finite learning rate
- Gradient descent might also get trapped at saddle points and/or local minima

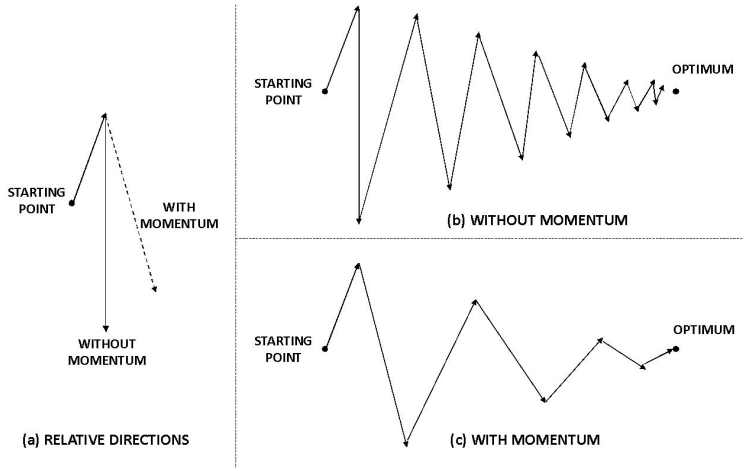
We need to :

– Move quickly in directions with small but consistent (pointing in one direction, +ve or -ve) gradients.

– Move slowly in directions with big but inconsistent (oscillating between –ve and +ve) gradients.

Gradient Descent with Momentum





- Add momentum to GD updates:

$$\overline{V} \Leftarrow \beta \overline{V} - \alpha \frac{\partial L}{\partial \overline{W}}; \quad \overline{W} \Leftarrow \overline{W} + \overline{V}$$

- Learning is accelerated as oscillations are damped and updates progress in the consistent directions of loss decrease
- Enables working with large learning rate values and hence faster convergence

Parameter-specific learning rates

- Apply a different learning rate to each parameter at each step
- Encourage faster relative movement in gently sloping direction
- Penalize dimension with large fluctuations in gradient
- Several optimizers: AdaGrad, RMSProp, RMSProp+Nestrov Momentum, AdaDelta, Adam
  - Differ in the manner parameter specific learning rates are calculated

Normalizing Input Data

- Min-max normalization (for feature j of input datapoint i)

$$x_{ij} \Leftarrow \frac{x_{ij} - min_j}{max_j - min_j}$$

  - Data with smaller standard deviation; scaled to be in the range [0,1]
  - Lessen the effect of outliers
- Standardization

$$x_{ij} \Leftarrow \frac{x_{ij} - mean_j}{std\_dev_j}$$

- Normalization helps in the convergence of optimization algorithm
- Should apply same normalization parameters to both train and test set
- Normalization parameters are calculated using train data
- Training converges faster when the inputs are normalized

Batch normalization

- Internal covariance shift – change in the distribution of network activations due to change in network parameters during training
- Idea is to reduce internal covariance shift by applying normalization to inputs of each layer
- Achieve fix distribution of inputs at each layer
- Normalization *for each training mini-batch*.
- Batch normalization enables training with larger learning rates
  - Reduces the dependence of gradients on the scale of the parameters
  - Faster convergence and better generalization

Batch normalization

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;
Parameters to be learned: γ, β

Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

$$\sigma^2_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma \widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

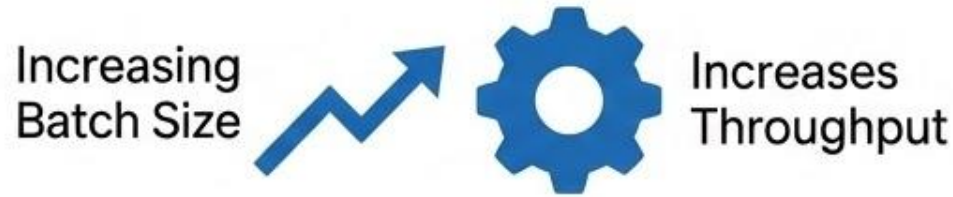
Why this step ?

Single Node, Single GPU Training

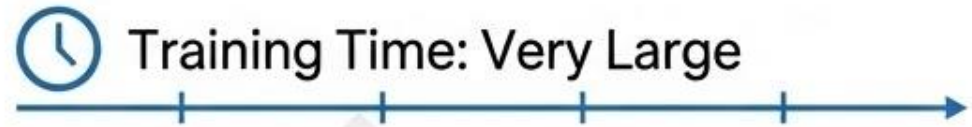
Throughput Dependencies

- Neural network model (activations, parameters, compute operations)
- Batch size
- Compute hardware: GPU type
- Floating point precision (FP32 vs FP16)
  - Using FP16 can reduce training times and enable larger batch sizes/models without significantly impacting the accuracy of the trained model

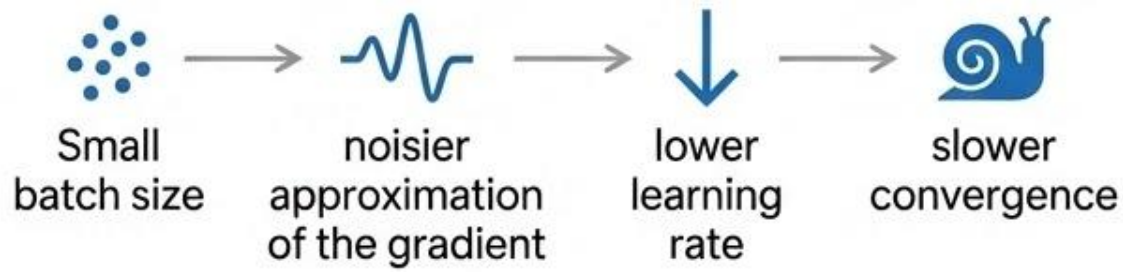
Key Considerations & Dynamics



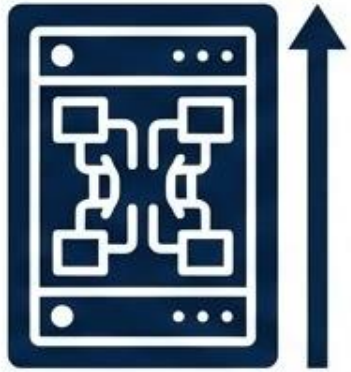
Batch size is restricted by GPU memory



Example: 6 days with Places dataset (2.5M images) using Alexnet on a single K40.



Multi-GPU Execution Scaling



Vertical scaling-up in a single node

NVIDIA servers (e.g., DGX-1 with 8 GPUs, DGX-2 with 16 GPUs) utilize **high-speed interconnects (like NVLink) between GPUs** within a single node.

Horizontal scaling-out across multiple nodes

Scales across multiple nodes, requiring **high-speed network interconnects (like InfiniBand) between servers**, common in supercomputers like Summit and Sierra.

High Performance Networking

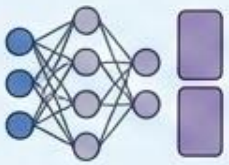
- Large Scale Parallel and Deep Learning applications needs:
  - High Bandwidth
  - Low Latency
- Ethernet is not enough
- Infiniband (IB) is widely adopted
- Custom Networks are the best

Network technology	Bandwidth [Gb/s]	Latency [us]
10GigE	10	4
40GigE	40	4
IB EDR	100	1
NVLink	> 400	0.1-0.2

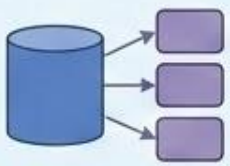
Distributed Training

Types of Parallelism

Model, Data, Hybrid



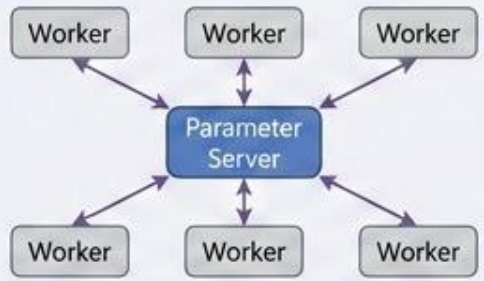
Model Parallelism



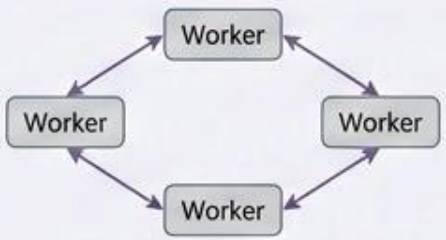
Data Parallelism

Hybrid Parallelism

Types of Aggregation



Centralized (Parameter Server)



Decentralized (P2P, All-reduce)

Centralized aggregation: parameter server;
Decentralized aggregation: P2P, all reduce

Performance Metric: Scaling Efficiency

$$\text{Scaling Efficiency} = \frac{\text{Runtime on 1 GPU}}{\text{Runtime on n GPUs}}$$

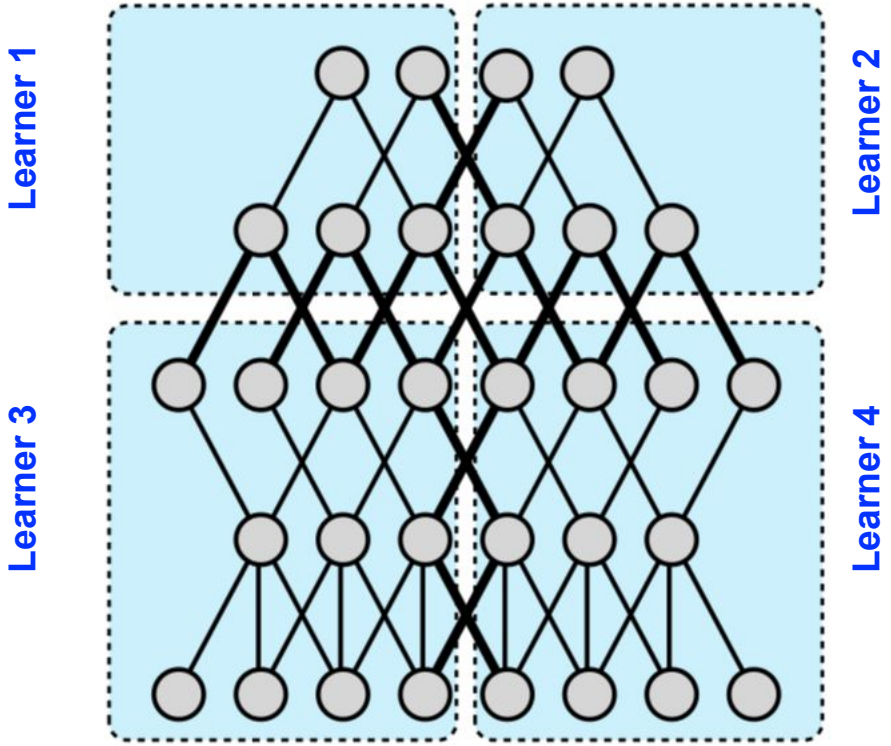
Defined as the ratio between the run time of one iteration on a single GPU and the run time of one iteration when distributed over n GPUs.

Parallelism

- Parallel execution of a training job on different compute units through scale-up (single node, multiple and faster GPUs) or scale-out (multiple nodes distributed training)
- Enables working with large models by partitioning model across learners
- Enables efficient training with large datasets using large “effective” batch sizes (batch split across learners)
  - Speeds up computation
- Model, Data, Hybrid Parallelism

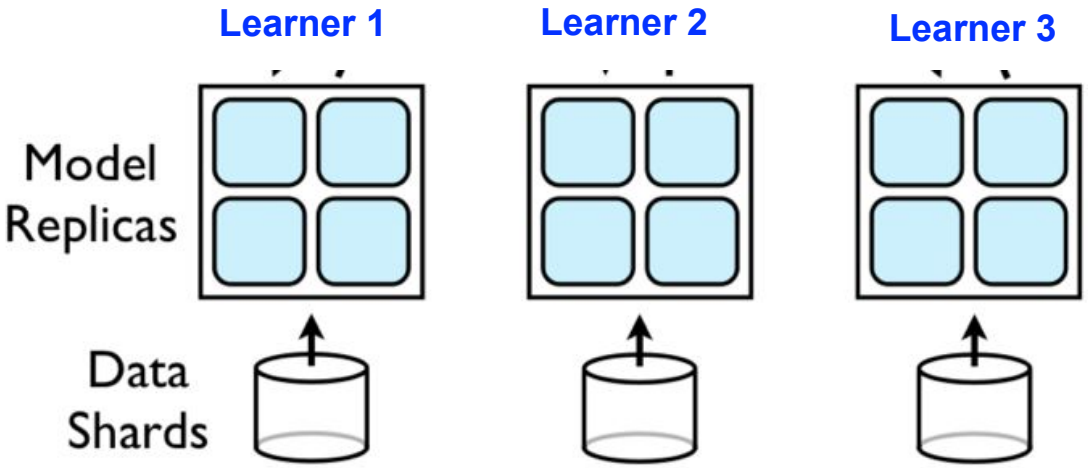
Model Parallelism

- Splitting the model across multiple learners



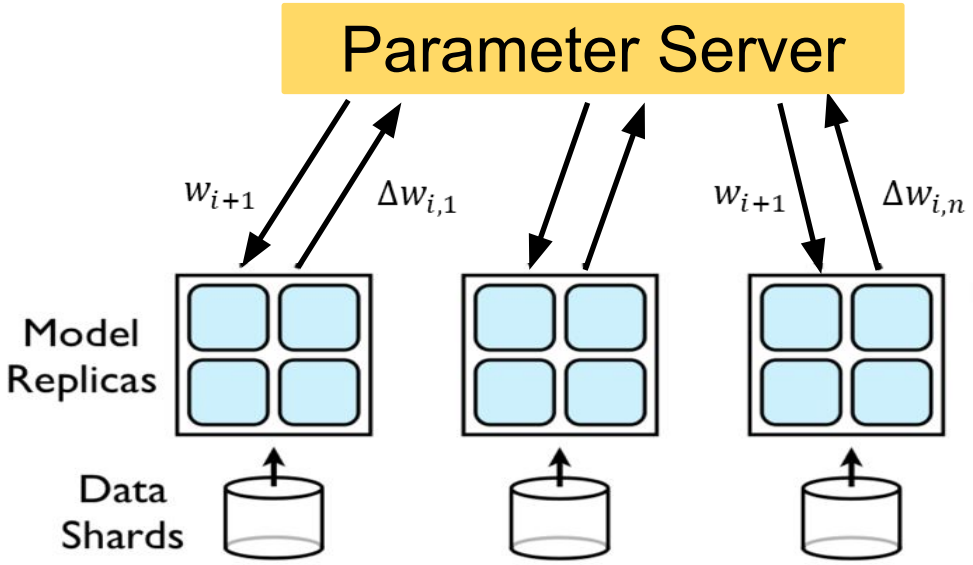
- 5 layered neural network
- Partitioned across 4 learners
- Bold edges cross learn boundaries and involve inter-learner communication
- Performance benefits depend on
  - Connectivity structure
  - Compute demand of operations
- Heavy compute and local connectivity –benefit most
  - Each machine handles a subset of computation
  - Low network traffic

Data Parallelism



- Model is replicated on different learners
- Data is sharded and each learner work on a different partition
- Helps in efficient training with large amount of data
- Parameters (weights, biases, gradients) from different replicas need to be synchronized

Parameter server (PS) based Synchronization



- Each learner executes the entire model
- After each mini-batch processing a learner calculates the gradient and sends it to the parameter server
- The parameter server calculates new value of weights and sends them to the model replicas

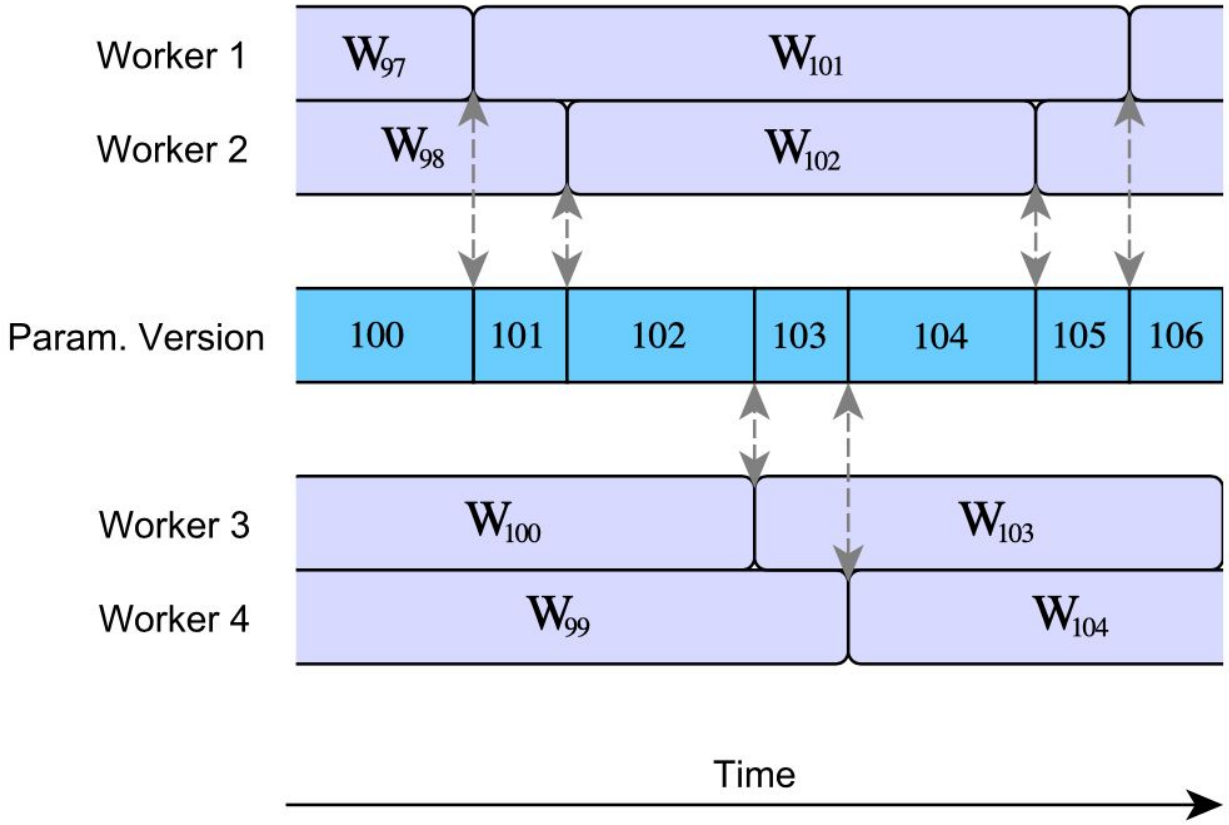
Synchronous SGD and the Straggler problem

- PS needs to wait for updated gradients from all the learners before calculating the model parameters
- Even though size of mini-batch processed by each learner is same, updates from different learners may be available at different times at the PS
  - Randomness in compute time at learners
  - Randomness in communication time between learners and PS
- Waiting for slow and straggling learners diminishes the speed-up offered by parallelizing the training

Asynchronous SGD and Stale Gradients

- PS updates happen without waiting for all learners
- Weights that a learner uses to evaluate gradients may be old values of the parameters at PS
  - Parameter server asynchronously updates weights
  - By the time learner gets back to the PS to submit gradient, the weights may have already been updated at the PS (by other learners)
  - Gradients returned by this learner are stale (i.e., were evaluated at an older version of the model)
- Stale gradients can make SGD unstable, slowdown convergence, cause sub-optimal convergence (compared to Sync-SGD)

Stale Gradient Problem in Async-SGD



$$\vdots$$

$$W_{101} = W_{100} - \lambda \Delta W_{97}$$

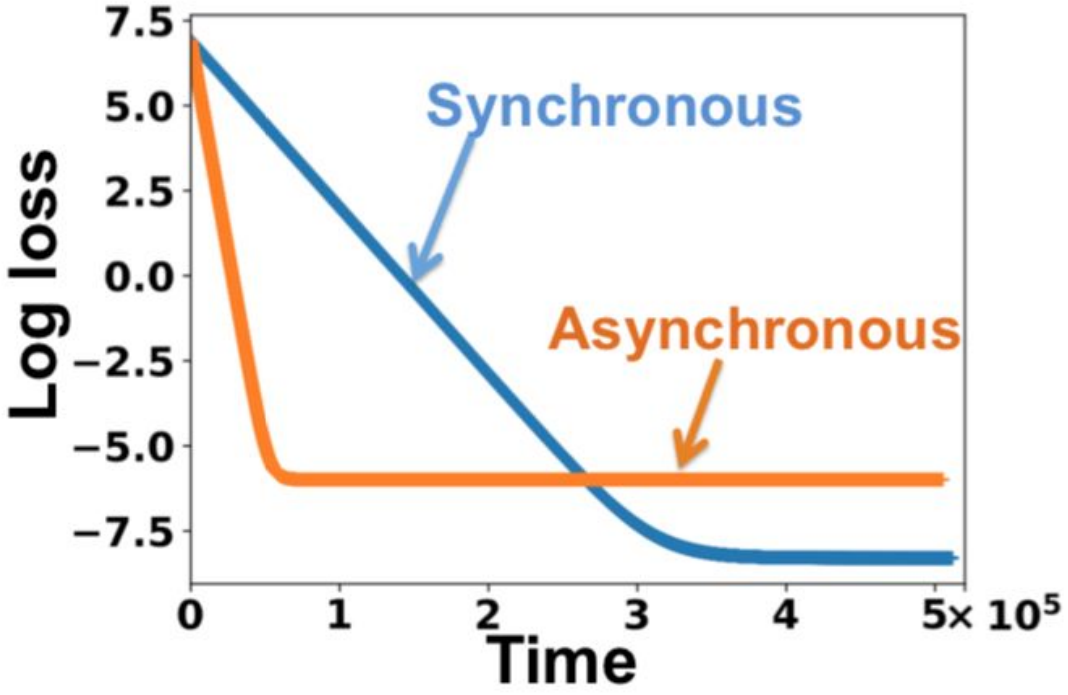
$$W_{102} = W_{101} - \lambda \Delta W_{98}$$

$$W_{103} = W_{102} - \lambda \Delta W_{100}$$

$$W_{104} = W_{103} - \lambda \Delta W_{99}$$

$$\vdots$$

Convergence-Runtime Tradeoff in SGD Variants



- Error-runtime trade-off for Sync and Async-SGD with same learning rate.
- Async-SGD has faster decay with time but a higher error floor.

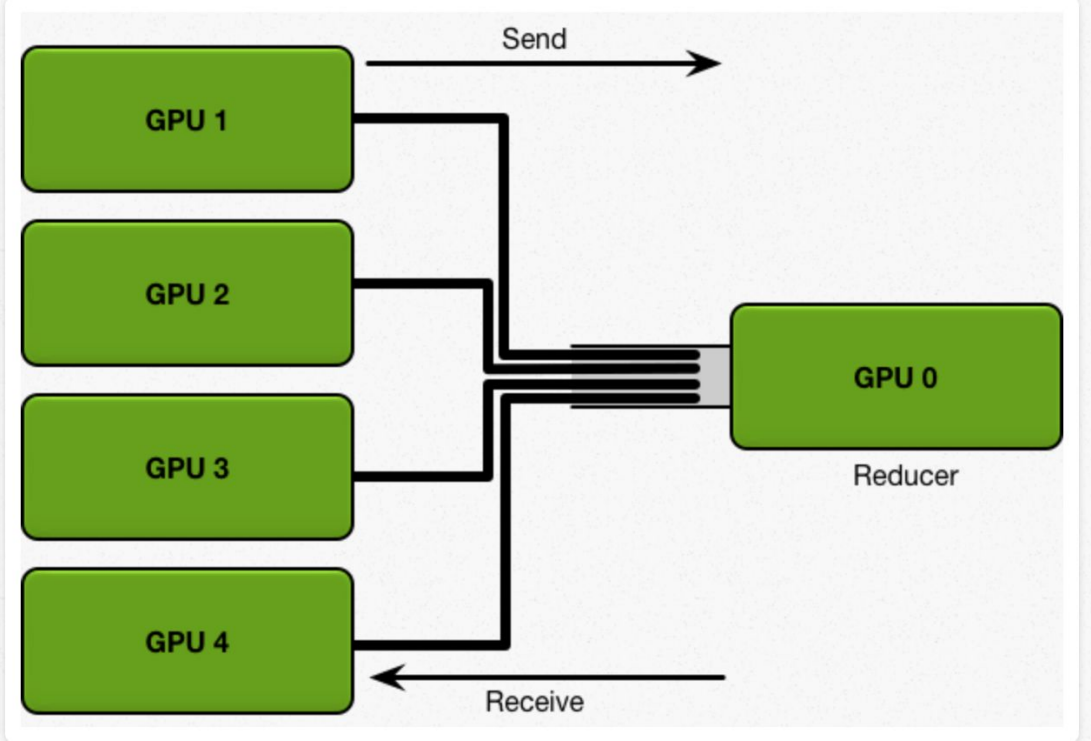
Reduction over Gradients

- To synchronize gradients of N learners, a reduction operation needs to be performed

$$\sum_{j=1}^{N} \Delta w_j$$

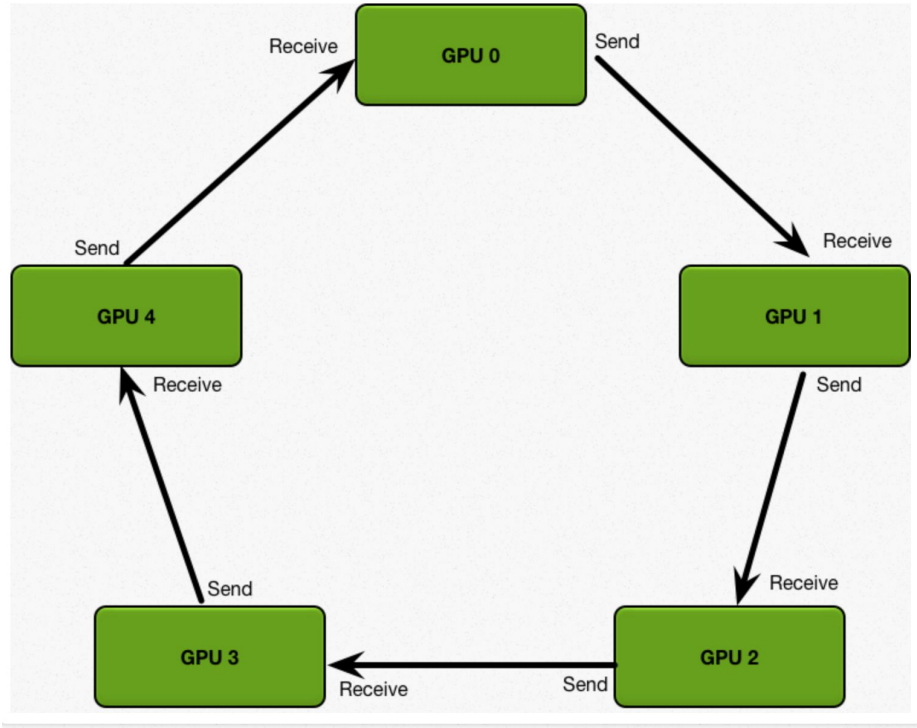
Reduction Topologies

Parameter server: single reducer



SUM (Reduce operation) performed at PS

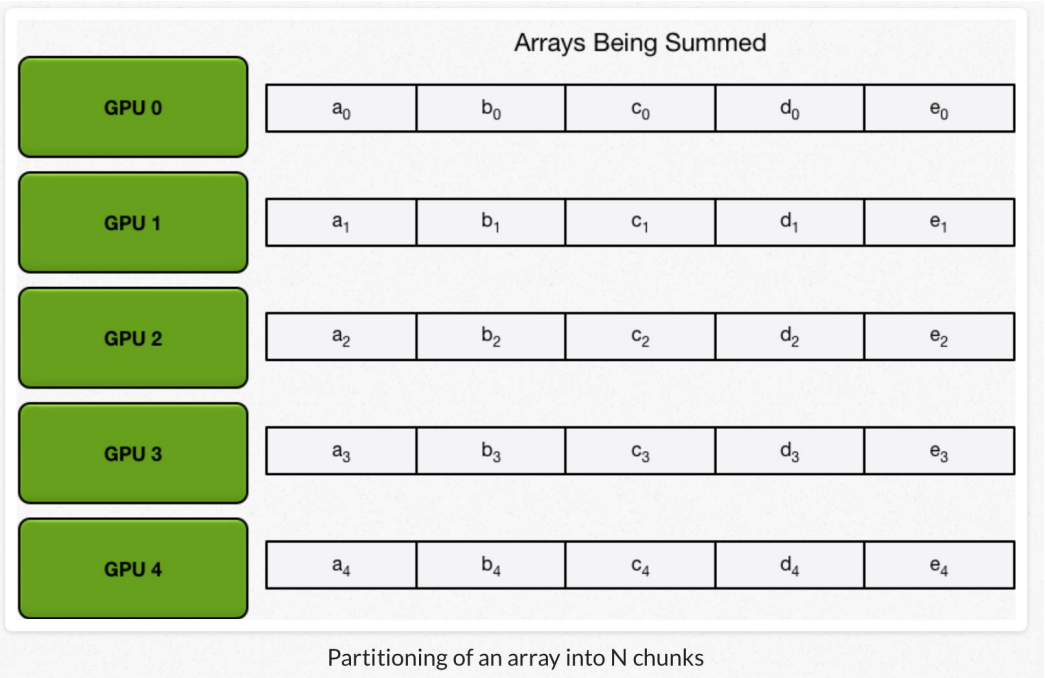
GPUs arranged in a logical Ring (aka bucket) :
all are reducers



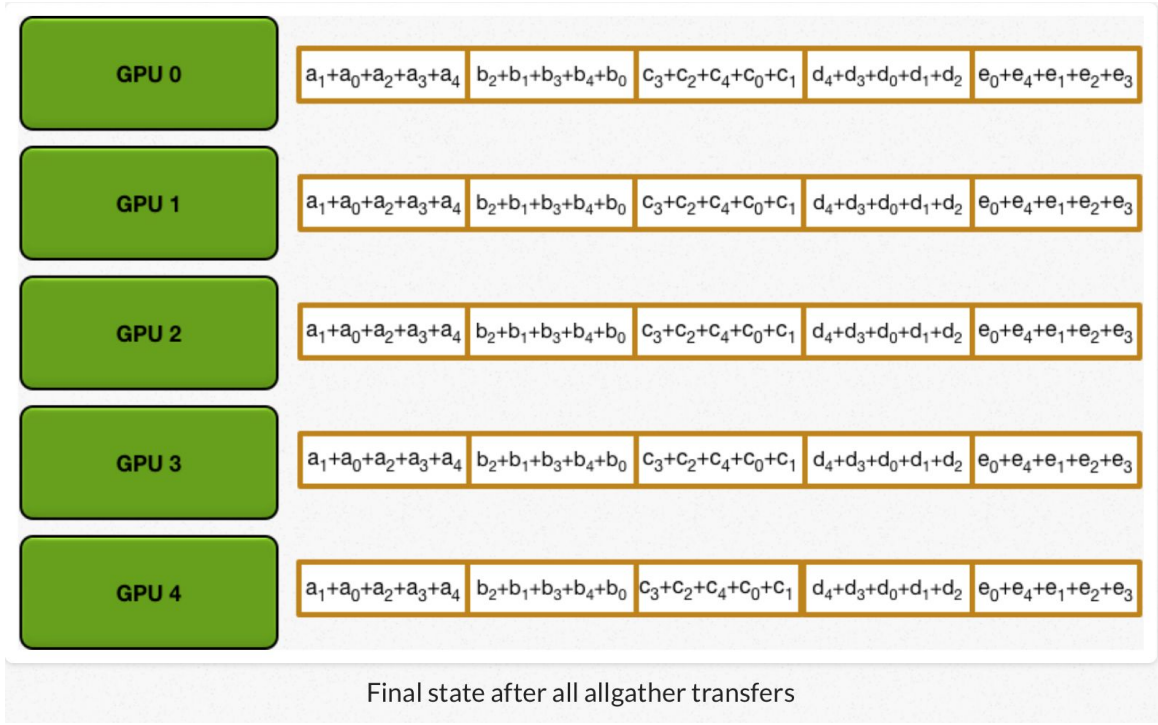
SUM (Reduce operation) performed at all nodes

- Each node has a left neighbor and a right neighbor
- Node only sends data to its right neighbor, and only receives data from its left neighbor

All-Reduce



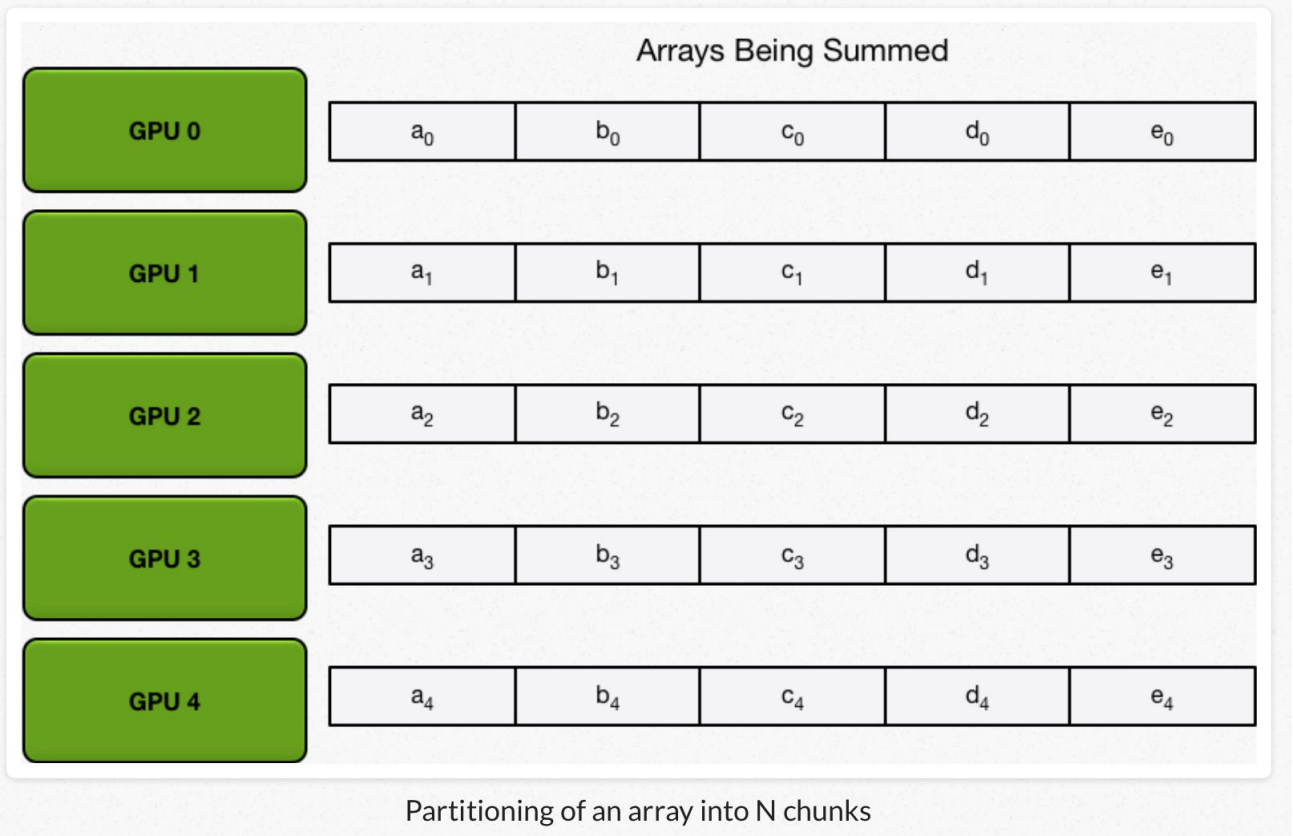
Partitioning of an array into N chunks



Final state after all allgather transfers

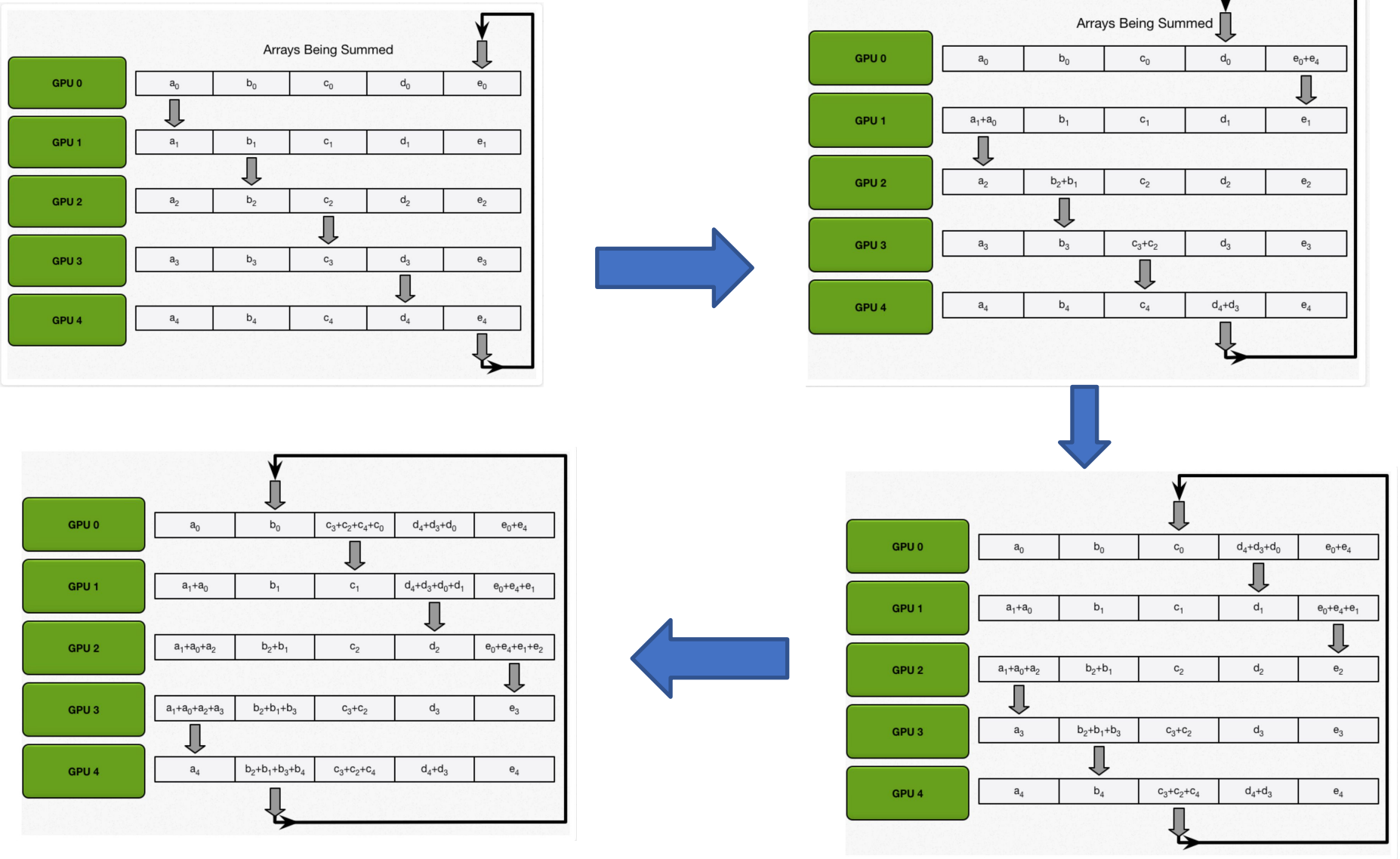
Ring All-Reduce

- Two step algorithm:
  - *Scatter-reduce*
    - GPUs exchange data such that every GPU ends up **with a chunk** of the final result
  - *Allgather*
    - GPUs exchange chunks from scatter-reduce such that all GPUs end up with the complete final result.

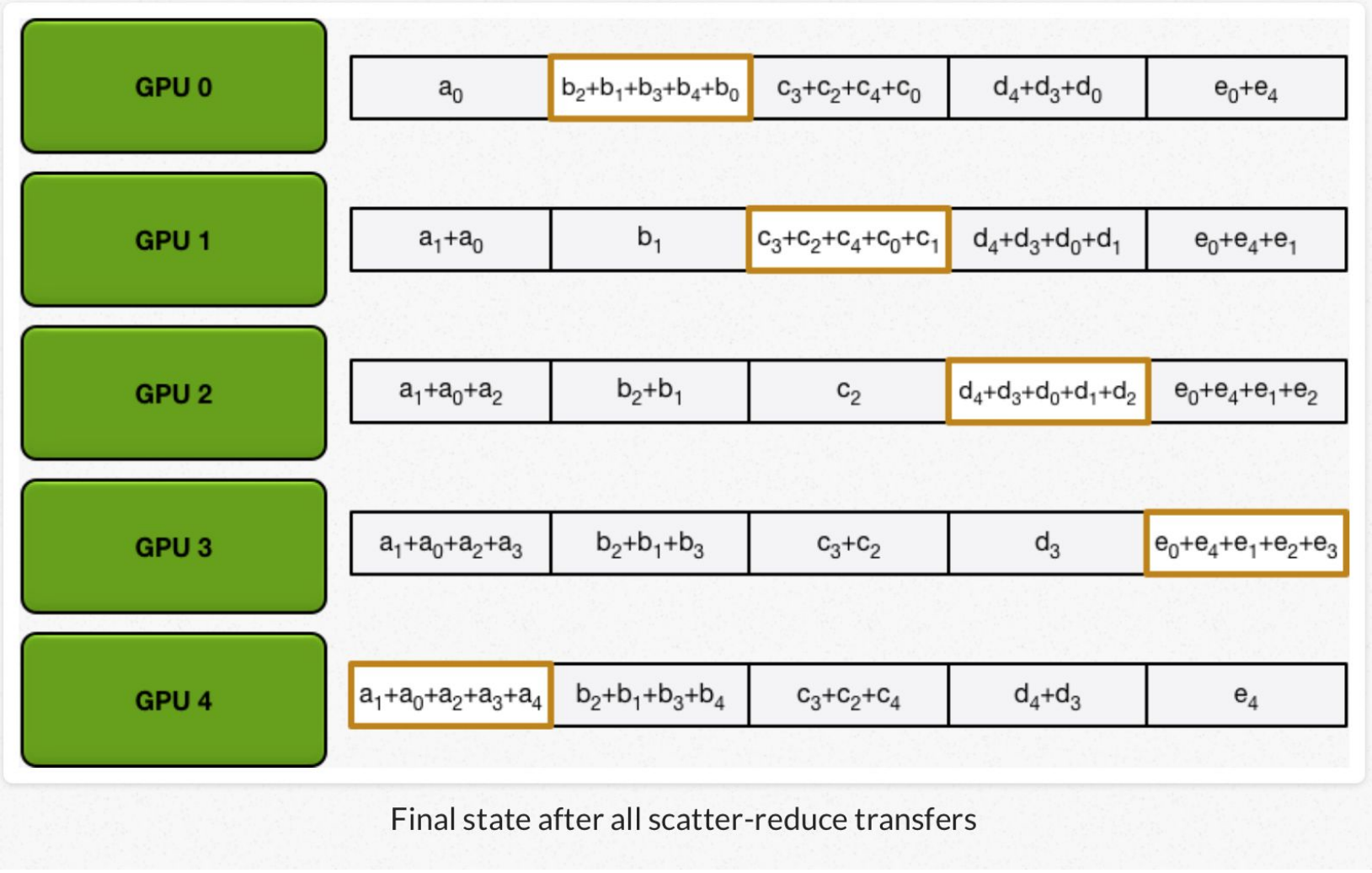


Partitioning of an array into N chunks

Ring All-Reduce: Scatter-Reduce Step



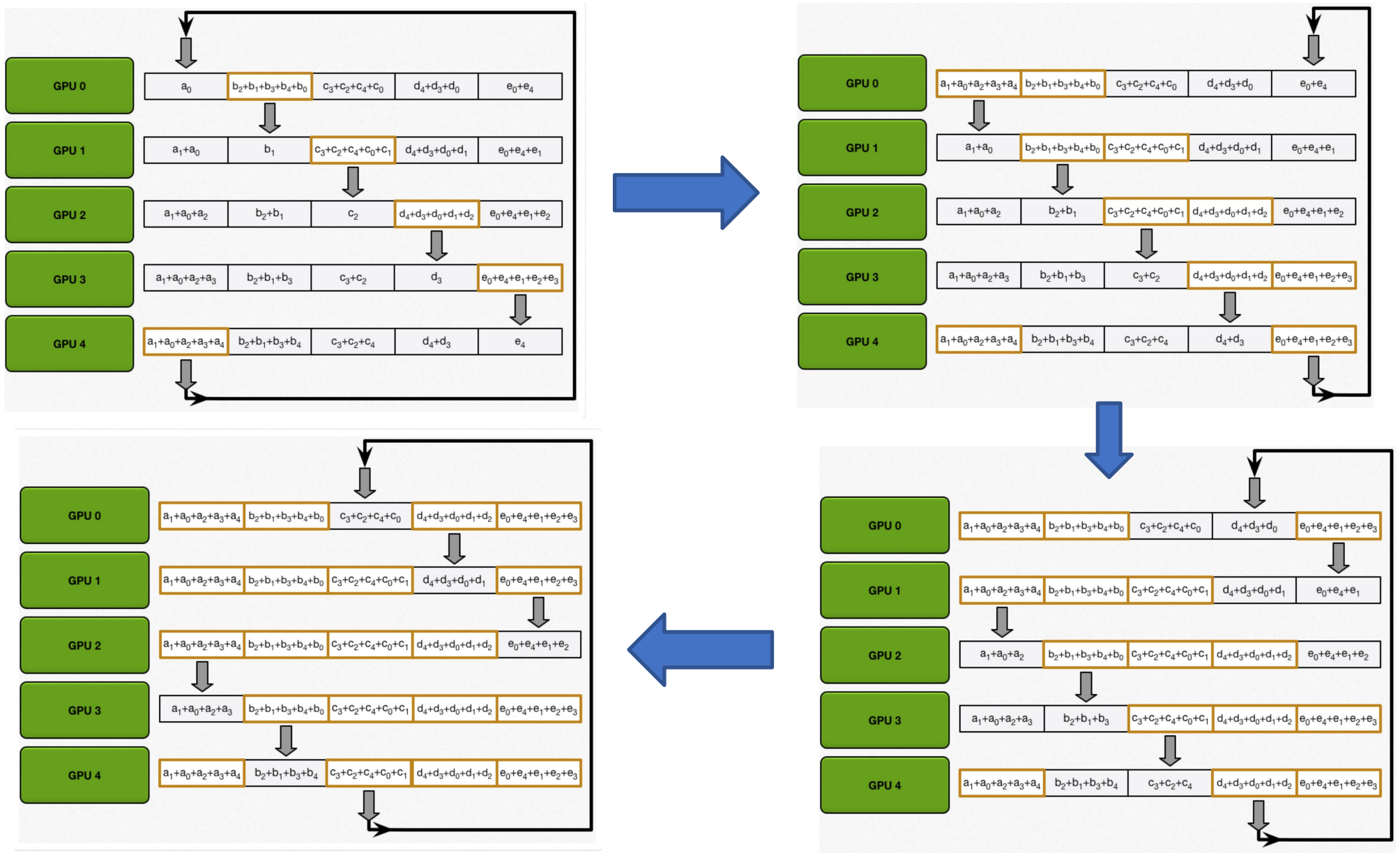
Ring All-Reduce: End of Scatter-Reduce Step



Final state after all scatter-reduce transfers

How many iterations in scatter-reduce step with N GPUs ?

Ring All-Reduce: AllGather Step



Ring All-Reduce: End of AllGather Step

Final state after all allgather transfers

How many iterations in allgather step with N GPUs ?

Parameter Server (PS) vs Ring All-Reduce: Communication Cost

- P: number of processes N: total number of model parameters
- PS (centralized reduce)
  - Amount of data sent to PS by (P-1) learner processes: N(P-1)
  - After reduce, PS sends back updated parameters to each learner
  - Amount of data sent by PS to learners: N(P-1)
  - Total communication cost at PS process is proportional to 2N(P-1)
- Ring All-Reduce (decentralized reduce)
  - Scatter-reduce: Each process sends N/P amount of data to (P-1) learners
    - Total amount sent (per process): N(P-1)/P
  - AllGather: Each process again sends N/P amount of data to (P-1) learners
  - Total communication cost per process is 2N(P-1)/P
- PS communication cost is proportional to P whereas ring all-reduce cost is practically independent of P for large P (ratio (P-1)/P tends to 1 for large P)
- Which scheme is more bandwidth efficient ?
- Note that both PS and Ring all-reduce involve synchronous parameter updates